

Instituto Politécnico Nacional
Escuela Superior de Cómputo




**Trabajo Terminal: 2026-A114**

---

# Simulación de sistemas complejos mediante autómatas celulares neuronales

---

*Presenta:*

**Angeles Lopez Erick Jesse**

eangelesl1700@alumno.ipn.mx

*Director:*

**Dr. Genaro Juárez Martínez**

Ciudad de México, 15 de abril de 2026

# Índice general

**1. Introducción** **1**

1.1. Resumen . . . . . 1

1.2. Planteamiento del problema . . . . . 1

1.2.1. Preguntas de investigación . . . . . 3

1.3. Objetivo . . . . . 3

1.3.1. Objetivos específicos . . . . . 4

1.4. Hipótesis . . . . . 4

1.5. Justificación . . . . . 4

1.6. Alcances y limitaciones . . . . . 6

**2. Marco teórico: Sistemas, caos y simulación** **7**

2.1. Sistemas . . . . . 8

2.1.1. Clasificación de sistemas . . . . . 9

2.1.1.1. Sistemas abiertos y cerrados . . . . . 9

2.1.1.2. Sistemas deterministas y estocásticos . . . . . 10

2.1.1.3. Sistemas estáticos y dinámicos . . . . . 10

2.2. Teoría del caos . . . . . 12

2.2.1. Atractores . . . . . 12

2.2.1.1. Tipos de atractores . . . . . 13
2.3. Sistemas complejos . . . . . 14
2.4. Modelado de sistemas complejos . . . . . 16

**3. Marco teórico: Autómatas celulares 19**
3.1. Autómata celular . . . . . 20
3.2. Vecindad . . . . . 21
3.3. Estados . . . . . 21
3.4. Condiciones de frontera . . . . . 24
3.5. Regla local . . . . . 26
3.6. Ejemplos . . . . . 28
3.6.1. John Von Neumann . . . . . 28
3.6.2. El juego de la vida . . . . . 29
3.6.3. Autómata celular elemental . . . . . 33
3.7. Clasificación . . . . . 34
3.8. Extensiones y cambios de la definición . . . . . 39

**4. Marco teórico: Inteligencia artificial y redes neuronales 41**
4.1. Inteligencia artificial . . . . . 42
4.2. Neurona biológica . . . . . 44
4.3. Neurona artificial . . . . . 44
4.4. Redes neuronales . . . . . 46
4.4.1. Funciones de activación . . . . . 47
4.4.1.1. Binary step (escalón binario) . . . . . 48
4.4.1.2. Lineal . . . . . 48

4.4.1.3. Sigmoide . . . . . 49
4.4.1.4. tanh (hiperbólica) . . . . . 50
4.4.1.5. ReLU (Rectified Linear Unit) . . . . . 50
4.4.2. Aprendizaje en redes neuronales . . . . . 51
4.4.2.1. Función de pérdida . . . . . 51
4.4.2.2. Optimización . . . . . 52
4.4.2.3. Entrenamiento . . . . . 53
4.5. Tipos de redes neuronales . . . . . 54
4.5.1. Redes neuronales convolucionales . . . . . 55
4.5.2. Redes neuronales recurrentes . . . . . 56

**Referencias** **58**

# Capítulo 1

# Introducción

## 1.1. Resumen

En el presente documento se presenta la primera etapa de una investigación orientada a la simulación de sistemas complejos en espacios bidimensionales mediante autómatas celulares neuronales. El objetivo principal es utilizar una red neuronal para definir la función de transición de un autómata celular, con el fin de modelar y simular las representaciones de sistemas complejos sobre una red regular bidimensional.

Los resultados incluyen el desarrollo del marco teórico, que recopila el conjunto de definiciones formales necesarias; el estado del arte, donde se analizan investigaciones previas relacionadas con diferentes enfoques y configuraciones; la descripción del marco metodológico, en donde se especifican las técnicas, herramientas y procedimientos utilizados; y los avances obtenidos hasta el momento, que abarcan la implementación y documentación de herramientas para la simulación de autómatas celulares en dos dimensiones.

## 1.2. Planteamiento del problema

La simulación de sistemas complejos ha surgido como una herramienta indispensable para la comprensión y el análisis de fenómenos en diversas áreas del conocimiento, desde la física y la biología hasta las ciencias sociales y económicas. Tradicionalmente, los autómatas celulares han servido como un modelo matemáti-

co fundamental para estudiar estos sistemas mediante la representación de procesos dinámicos.

Los autómatas celulares son un modelo matemático definido como una 4-tupla que contiene:

- Un espacio de celdas en un dominio discreto.
- Un conjunto de estados posibles.
- Una vecindad que define las dependencias entre células.
- Una función de transición que determina cómo evolucionan simultáneamente las células [1, 2].

Ejemplos clásicos de autómatas celulares incluyen el Juego de la Vida de John Conway y los 256 autómatas elementales descritos por Stephen Wolfram. Estas herramientas han sido aplicadas en múltiples contextos como la simulación de incendios, modelado de distribuciones poblacionales, simulación de partículas y el análisis de ciclos climáticos [3]. No obstante, una limitación crítica de los modelos convencionales reside en la rigidez de sus funciones de transición; estas deben definirse de manera estática y anticipada, restringiendo la posibilidad de adaptarse a condiciones imprevistas o cambios en el entorno. Esta limitación motiva la búsqueda de nuevas metodologías capaces de incorporar mayor flexibilidad y adaptabilidad en la simulación de sistemas dinámicos.

En este contexto, se busca la integración de redes neuronales en el marco de los autómatas celulares. Las redes neuronales, inspiradas en el funcionamiento del cerebro y su capacidad de aprendizaje, ofrecen la posibilidad de redefinir las reglas de transición de forma dinámica. Esto permite que el autómata celular se ajuste de manera autónoma a partir de la experiencia y el entrenamiento [4].

El presente proyecto de investigación propone superar las limitaciones de los autómatas celulares mediante la integración de redes neuronales en la definición de las reglas de transición, dando origen a los *autómatas celulares neuronales*. En este enfoque, las reglas de actualización dejan de ser fijas y adquieren la capacidad de aprender y adaptarse a partir de los datos, lo que permite que el sistema se ajuste de forma dinámica ante comportamientos complejos y entornos variables [5, 6]. Esta aproximación no solo busca ampliar el campo de aplicación de los autómatas celulares, sino que también busca abrir nuevas oportunidades para modelar y simular fenómenos de formas realistas.

Dadas las infinitas configuraciones de autómatas celulares, este proyecto se limita a un espacio bidimensional finito (con condiciones de frontera) y un conjunto finito y numerable de estados. Adicionalmente, se busca experimentar con diferentes vecindades, incluyendo Moore y Neumann. Aunque el objetivo no es determinar las condiciones para la evolución de las células, sí se busca realizar pruebas bajo diferentes configuraciones que permitan mostrar los resultados sin comprometer su calidad.

Por otro lado, la red neuronal recibe como parámetros los estados de las celdas vecinas y devuelve el nuevo estado de la celda a la cual se aplique la función de transición. Este proceso es el mismo para todas las células.

Es por esto que surge la pregunta central de esta investigación: ¿Cómo los autómatas celulares neuronales pueden simular sistemas complejos en un espacio bidimensional? A partir de este cuestionamiento, surgen las siguientes preguntas de investigación.

### 1.2.1. Preguntas de investigación

- ¿Qué es un sistema complejo?
- ¿Qué es simular?
- ¿Cómo se simula un sistema complejo?
- ¿Qué es un autómata celular?
- ¿Qué es una red neuronal?
- ¿Qué es un autómata celular neuronal?
- ¿Cómo simular un sistema complejo?

## 1.3. Objetivo

Desarrollar un sistema de simulación basado en autómatas celulares neuronales bidimensionales que permita analizar la generación, mantenimiento, recuperación y propagación de patrones de información en entornos complejos y dinámicos.

### 1.3.1. Objetivos específicos

- Diseñar e implementar un simulador de espacios discretos bidimensionales que sirva de plataforma para estudiar los autómatas celulares neuronales.
- Entrenar y evaluar el desempeño de los autómatas celulares neuronales bajo diferentes configuraciones.
- Replicar configuraciones clásicas con comportamientos y soluciones conocidas.
- Analizar y comparar diferentes comportamientos con base en la red neuronal establecida.

## 1.4. Hipótesis

Si un sistema de simulación está basado en autómatas celulares neuronales bidimensionales, entonces es posible analizar y reproducir la generación, mantenimiento y propagación de patrones de información en entornos complejos y dinámicos.

## 1.5. Justificación

Los autómatas celulares son modelos lo bastante simples para permitir un análisis teórico riguroso, pero lo suficientemente complejos para exhibir fenómenos emergentes. Definir las reglas que describen comportamientos erráticos o no lineales de forma sincronizada resulta una tarea compleja [7].

Por esta razón, este proyecto de investigación nace de la necesidad de superar las limitaciones de los modelos tradicionales de autómatas celulares, que se basan en reglas de transición fijas y ya establecidas. Este enfoque limitado, aunque ha sido útil para entender algunos fenómenos, no siempre refleja la complejidad y la capacidad de adaptación de los sistemas dinámicos que encontramos en la naturaleza, en la sociedad y en la tecnología.

Cada autómata celular resuelve un problema particular; sin embargo, su definición matemática implica la existencia de una infinidad de posibles configuraciones y comportamientos. Por ello, llevar a cabo ajustes manuales hasta obtener el resultado deseado se convierte en un proceso computacionalmente inviable [8].

Se sugiere usar redes neuronales para definir las reglas de transición, con el fin de crear un modelo de autómata celular neuronal que pueda aprender y adaptarse por sí mismo a los cambios en su entorno. Esto no solo amplía el alcance y la aplicabilidad de los autómatas celulares, sino que también abre un novedoso campo de estudio en el que se combinan herramientas de inteligencia artificial y simulación de sistemas complejos.

La principal innovación de este enfoque reside en reemplazar las funciones de transición estáticas por modelos ajustables. Usando técnicas de aprendizaje automático, se puede entrenar la función de transición con datos representativos. Esto permite que el sistema se ajuste por sí solo a los cambios en el entorno [9].

La elección de redes neuronales para modelar la función de transición no es arbitraria. Varios estudios recientes han mostrado que las redes neuronales, sobre todo las convolucionales y recurrentes, son muy efectivas para aproximar funciones no lineales y entender relaciones complejas en el espacio y el tiempo [10, 11]. Estas características las hacen particularmente adecuadas para emular las dinámicas locales y temporales propias de los autómatas celulares. Este proyecto se basa en una línea de investigación establecida que intenta unir la capacidad expresiva de las redes neuronales con la estructura organizada y distribuida de los autómatas celulares. Así, se busca crear un modelo más flexible y que se pueda aplicar en diferentes situaciones.

Adicionalmente, este proyecto profundiza en técnicas de simulación, modelado matemático y redes neuronales, integrando conocimientos de ciencias de la computación, vida artificial y matemáticas.

La experiencia adquirida durante el desarrollo e implementación de estos modelos abrirá puertas a futuras investigaciones y proyectos en el campo de la inteligencia artificial. Además, la creación de un entorno de código abierto contribuirá al avance del estado del arte y facilitará que otros investigadores puedan extender y aplicar estos conceptos.

En resumen, este proyecto tiene como objetivo ayudar en el desarrollo de la simulación de sistemas complejos. Para lograrlo, se creará un modelo de autómatas celulares neuronales que sea flexible, adaptable y que se pueda ampliar. Esto unirá la teoría de sistemas dinámicos con las técnicas actuales de aprendizaje automático.

## 1.6. Alcances y limitaciones

### Alcances

- El estudio se limita a la simulación en espacios bidimensionales finitos con condiciones de frontera establecidas en una red regular definida.
- Se emplean redes neuronales supervisadas para la definición de la función de transición.
- Se analizarán comportamientos de patrones conocidos y emergentes bajo distintas configuraciones de estados y vecindades.

### Limitaciones

- No se contempla la generalización del modelo a diferentes dimensionalidades o topologías de red.
- El análisis se centra en la simulación de sistemas complejos, sin abordar la optimización computacional del entrenamiento.
- Los resultados se interpretan únicamente dentro del contexto de los parámetros y condiciones experimentales definidos.

# Capítulo 2

# Marco teórico: Sistemas, caos y simulación

Estudiar fenómenos naturales y artificiales requiere el diseño de estructuras que permitan simplificar la realidad sin perder su esencia. En el presente capítulo se establecen los fundamentos teóricos para comprender la simulación computacional de sistemas, abarcando desde la Teoría General de Sistemas hasta la caracterización de los sistemas complejos.

En primer lugar, se explora la Teoría General de Sistemas como un enfoque interdisciplinario utilizado para comprender la complejidad en diferentes contextos. Esta perspectiva se apoya en dos pilares fundamentales: la concepción de que cualquier fenómeno puede ser entendido como un sistema y la premisa de que las conexiones entre las partes son esenciales para su funcionamiento.

Luego, se presenta la teoría del caos y el concepto de atractores, elementos que permiten diferenciar y analizar sistemas deterministas que presentan comportamientos aparentemente aleatorios.

Finalmente, dado que resulta impráctico experimentar directamente con sistemas complejos en la naturaleza, se justifica el uso del modelado y la simulación computacional como herramientas fundamentales para la experimentación y la inferencia científica.

## 2.1. Sistemas

La teoría general de sistemas es una forma ordenada de aproximación o simulación del mundo real con base en un procedimiento científico. Su objetivo es difundir una terminología general que pueda describir las características, funciones y comportamientos de cualquier sistema. No busca dar soluciones a problemas, sino que busca formular, matemáticamente, las teorías y definiciones conceptuales que se aplican a la realidad [12, 13].

De acuerdo con [14, 15, 16] la teoría general de sistemas establece tres principios básicos: (1) Los sistemas no son fenómenos aislados; un sistema siempre pertenece a otro sistema, por lo tanto (2) todos los sistemas son abiertos, y (3) las funciones de un sistema dependen únicamente de su estructura.

Un sistema es un conjunto de partes o "subsistemas" interrelacionados que actúan de forma coordinada, dicho de otro modo:

**Definición 1.** *Un sistema es la abstracción de un evento que existe en la naturaleza, que tiene un objetivo para el observador y cumple con los axiomas que lo caracterizan.*

Es decir, un sistema es un modelo matemático cuyas funciones están limitadas por su estructura, la cual recibe una entrada, realiza una función y devuelve una salida. Un modelo es cualquier abstracción de un proceso realizado por el observador y un proceso es cualquier fenómeno presente en la naturaleza que tenga un objetivo.

Los sistemas, y en general los modelos, son abstracciones inherentes al proceso original, es decir, un sistema puede referenciar múltiples procesos que tengan ciertas propiedades en común. Por ejemplo, el modelo que examina las fuerzas que interactúan en un objeto en caída libre no depende de especificaciones del objeto como su color o su forma.

El sistema permite probar las respuestas de un proceso a estímulos de interés para el observador. Siguiendo el ejemplo de caída libre, no es necesario soltar diferentes objetos de diferentes alturas para identificar las fuerzas que están interactuando; únicamente es necesario cambiar las entradas del modelo.

De acuerdo con [17], los sistemas se caracterizan principalmente por

- Estar compuestos por elementos internos independientes que pueden organizarse en subsistemas.

- Operar de manera organizada para cumplir un objetivo.
- Tener elementos que se puedan distinguir como parte del sistema o del ambiente.
- Existir en un marco temporal.

Los elementos que constituyen un sistema pueden ser materia, energía o información. Los dos primeros se encuentran regidos por las leyes de conservación. Por otro lado, la información se calcula como la cantidad ya existente más la información entrante y la generada, es decir, la información no se pierde [13].

### 2.1.1. Clasificación de sistemas

De acuerdo con [13], los sistemas se categorizan según su entitividad, origen y relación con el ambiente. Complementando esta visión, diferentes autores añaden clasificaciones con base en el dominio, evolución, continuidad, origen, etc. A continuación se listan algunas de las clasificaciones que resultarán relevantes para la definición de sistemas complejos.

#### 2.1.1.1. Sistemas abiertos y cerrados

Un sistema es abierto si mantiene un intercambio permanente de elementos con su entorno [13]. Tienen una interacción activa con el mundo que los rodea o con su ambiente. Estas influencias externas pueden ajustar las respuestas del propio sistema y se caracterizan por:

- Adaptabilidad: Pueden ajustar el comportamiento y las estructuras internas para mantener su funcionamiento y equilibrio en respuesta a perturbaciones externas.
- Homeostasis dinámica: Pueden mantener un equilibrio interno a lo largo del tiempo.
- Entradas y salidas: Sus entradas adquieren información del entorno y las salidas le devuelven una respuesta [18].

Por su parte, los sistemas cerrados se encuentran aislados de su entorno. Tiene límites que actúan como barreras para el intercambio. Estos operan de forma autocontenida y no tienen una interacción con el entorno que los rodea.

Siguiendo a [18], todas las fluctuaciones y cambios dependen únicamente de los elementos que se encuentren dentro del sistema. Estos sistemas tienden a mantener su estructura y equilibrio interno de manera constante; esto se debe principalmente a que:

- La frontera impide interacción significativa con el entorno, por lo que no se ve afectado el funcionamiento interno.
- Conservan la energía y la materia que los contienen, lo que contribuye a la estabilidad de la estructura y el equilibrio.

Con base en los principios básicos de un sistema, es imposible que existan sistemas cerrados porque un sistema siempre pertenece a otro. La clasificación de sistemas según su relación con el ambiente busca disminuir esa interacción con el entorno para propiciar condiciones ideales de aislamiento.

#### 2.1.1.2. Sistemas deterministas y estocásticos

Un sistema determinista es aquel en donde las mismas entradas producirán invariablemente las mismas salidas. No se contempla la existencia del azar, variables aleatorias ni principio de incertidumbre.

Por otro lado, un sistema estocástico es aquel que se rige por el azar. El resultado depende de probabilidades que cambian o pueden variar en función del tiempo [19].

No hay que confundir los sistemas estocásticos con los sistemas caóticos. En los sistemas estocásticos existe una variable aleatoria que impide que las mismas entradas tengan las mismas salidas. Los sistemas caóticos son deterministas, pero establecer el sistema en evoluciones futuras resulta complicado.

#### 2.1.1.3. Sistemas estáticos y dinámicos

Un sistema estático o sin memoria es aquel que permanece en un estado constante o inmutable, es decir, no cambia con el tiempo. Se dice entonces que un sistema es estático si su salida en cualquier instante de tiempo depende solamente de entradas en ese mismo momento y, de acuerdo con [20], estos se caracterizan por:

- Inmutabilidad: No cambia con el tiempo y mantiene su estado constante.
- Estabilidad: Es sencillo predecir y controlar su comportamiento.
- Simplicidad: Usualmente son más sencillos en diseño y operación, lo que facilita su implementación.

Por otro lado, un sistema dinámico o con memoria es aquel que cambia con el tiempo. Estos tienen la capacidad de adaptarse y responder ante diferentes condiciones de entrada. En otras palabras, es aquel cuya salida en un instante de tiempo depende tanto de entradas en otros y en ese mismo momento. Este se caracteriza por:

- Cambio en el tiempo: Se adapta ante diferentes situaciones y entradas.
- Flexibilidad: Responde ante diferentes condiciones y requisitos.
- Complejidad: Son más complejos en diseño y operación debido a su capacidad de cambio.

De acuerdo con [21], no podemos encontrar procesos estáticos en la naturaleza, pero en ocasiones resulta impráctico modelarlos como sistemas dinámicos.

Este es un ejemplo de un sistema estático:

$$y[n] = 2x[n] + 2$$

Se considera estático porque en cualquier instante $n$, la salida $y[n]$ depende solamente del valor de entrada $x[n]$ en ese mismo momento.

Por otro lado, este es un ejemplo de un sistema dinámico:

$$y[n] = x[n] + x[n-1]$$

Ya que el valor de $y[n]$ en el instante $n$ requiere conocer el valor actual $x[n]$ y el valor previo $x[n-1]$.

La evolución de un sistema se lleva a cabo con base en estados: el sistema comienza en un estado $x_0$ y la evolución al estado siguiente depende del estado anterior; además, existe una ley que rige su evolución, y, conociendo esta ley, es posible determinar cualquier estado posterior [22].

## 2.2. Teoría del caos

De acuerdo con [23], el caos trata con sistemas cuyo comportamiento es impredecible a largo plazo. Suele ser asociado a sistemas aleatorios o que no tienen un orden, pero en teoría del caos se refiere a todos aquellos sistemas deterministas cuya predicción en instantes de tiempo futuro se vuelve difícil de calcular.

Las condiciones iniciales suelen contener errores que incrementarán exponencialmente con el tiempo, lo que se conoce como sensibilidad a las condiciones iniciales, haciendo imposible la predicción a largo plazo. Estos se conocen como sistemas caóticos.

**Definición 2.** *Sea $X$ un espacio métrico y sea $\phi : X \to X$ una aplicación continua. Se dice que un sistema es caótico si cumple las siguientes condiciones:*

- *La aplicación $\phi : X \to X$ es topológicamente transitiva.*
- *El conjunto de puntos de $\phi$ es denso en el atractor.*
- *La aplicación $\phi : X \to X$ tiene dependencias sensitivas a las condiciones iniciales.*

El hecho de que la aplicación sea topológicamente transitiva implica que el atractor caótico es irreducible, es decir, si iteramos la función $\phi$ un número determinado de veces (evolucionar el sistema durante un periodo de tiempo suficiente), se puede acceder a cualquier punto del atractor desde cualquier otro.

Si el conjunto de puntos es denso en el atractor, entonces cualquier punto en el espacio está arbitrariamente cerca de una órbita periódica.

Finalmente, la sensibilidad a las condiciones iniciales refleja el hecho de que en un sistema caótico, dos trayectorias que inician relativamente cerca acabarán comportándose de una forma diferente a largo plazo.

### 2.2.1. Atractores

En un espacio disipado, la región del espacio de fases que atrae a las trayectorias vecinas se conoce como atractor. Es un subconjunto en el espacio de fases, es decir, un conjunto de estados a los que tiende a evolucionar el sistema en función de las condiciones iniciales.

**Definición 3.** *Sea $X$ el espacio de fases de un sistema dinámico disipado y sea $\phi_t : X \to X$ la transformación que describe el movimiento. Se dice que $A \subset X$ es un atractor si verifica las siguientes condiciones:*

- *$\exists U$ entorno de $A$, llamado cuenca de atracción de $A$, tal que $\forall V$ abierto con $A \subset V$, existe un $T_0$ tal que $\phi_t(U) \subset V, \forall t > T_0$.*
- *$\phi_t(A) = A, \forall t > 0$*
- *$\nexists S \subset A$ subconjunto propio no vacío que satisfaga las dos propiedades anteriores.*

Es decir, un punto que se encuentra en el atractor no puede salir de ahí y existe una órbita alrededor de este en donde, para cualquier tiempo, la trayectoria permanece en la órbita.

#### 2.2.1.1. Tipos de atractores

A continuación, se describen los cuatro tipos de atractores que se encuentran en la figura 2.1.

1. Atractores de punto fijo: Un atractor es de punto fijo si existe un punto fijo tal que todas las trayectorias cercanas convergen a el cuando $t \to \infty$.

   La figura 2.1a muestra el comportamiento de un péndulo con resistencia al viento que responde a la ecuación:

   $$\theta'' + \gamma\theta' + \beta\theta'|\theta'| + \frac{g}{L}\sin(\theta) = 0$$

   La energía se disipa por la fricción $\gamma$ y por la resistencia del aire $\beta$, obligando al sistema a perder velocidad hasta detenerse en el origen.

2. Atractores de ciclo límite: Las trayectorias vecinas se acercan o se alejan del ciclo límite en forma de espiral.

   En la figura 2.1b se muestra el oscilador de Van der Pol, cuya dinámica esta definida por:

   $$x'' - \mu(1 - x^2)x' + x = 0$$

   El sistema aumenta la energía cuando la amplitud es pequeña y la disipa si es grande, por lo que todas las trayectorias convergen a una órbita periódica estable.

3. Atractores cuasiperiódicos: Cuando dos ciclos límites con diferentes trayectorias se superponen, estas se enroscan en las direcciones longitudinales y latitudinales, creando una figura toroidal.

   La Figura 2.1c ilustra un oscilador lineal forzado con una frecuencia externa basada en la razón áurea $\phi$:

   $$x'' + x = A\cos(\phi t)$$

   Debido a que $\phi$ es irracional respecto a la frecuencia del oscilador, el movimiento resultante es una superposición de trayectorias que nunca se repiten exactamente.

4. Atractores extraños: Se dice que un atractor es extraño si tiene dimensión fractal (patrón de autosimilitud a diferentes escalas). Además, un atractor extraño es caótico si las trayectorias del sistema son atraídas hacia él, pero dentro del atractor son inestables. Esto significa que son sensibles a las condiciones iniciales.

   La Figura 2.1d muestra el atractor de Rössler, definido por el siguiente sistema de ecuaciones:

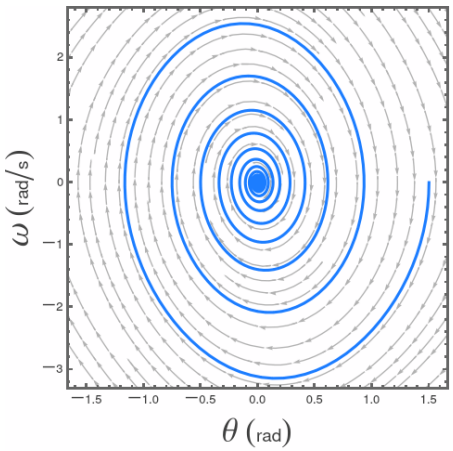
   $$\begin{cases} x' = -y - z \\ y' = x + ay \\ z' = b + z(x - c) \end{cases}$$

   Las trayectorias se atraen hacia una estructura geométrica compleja donde se estiran y se pliegan continuamente, lo que genera un comportamiento impredecible a largo plazo a pesar de ser un sistema determinista.

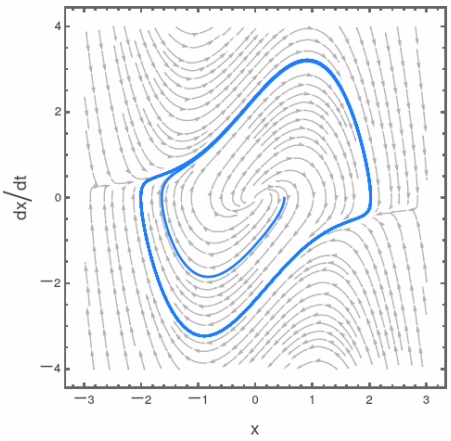
## 2.3. Sistemas complejos

Los sistemas complejos son sistemas dinámicos con muchas componentes que interactúan entre sí mediante el intercambio de materia, energía o información y que, eventualmente, sufren la acción de agentes externos [24]. Las colonias de hormigas, el mercado bursátil, el clima, el sistema nervioso, las células y los organismos vivos son algunos ejemplos de sistemas complejos. Estos se caracterizan por tener un conjunto de elementos que interactúan entre sí con el fin de alcanzar un objetivo [25, 26].

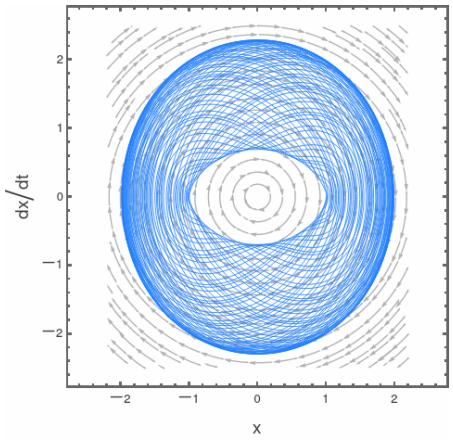
Los sistemas complejos tienen dos implicaciones fundamentales: (1) Existe una influencia mutua entre sus elementos, donde el cambio afecta inevitablemente al



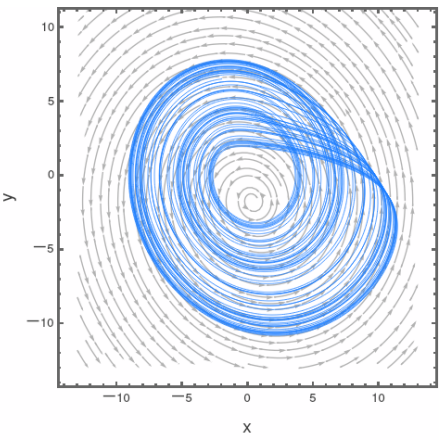

(a) Punto fijo: Convergencia al reposo en un péndulo amortiguado.




(b) Ciclo límite: Oscilador de Van der Pol.




(c) Cuasiperiódico: Trayectoria densa generada por frecuencias inconmensurables.




(d) Extraño: Atractor de Rössler que exhibe una estructura fractal y caos.

Figura 2.1: Representación de los cuatro tipos principales de atractores en el espacio de fases.

resto de elementos, y (2) persiguen un objetivo [26]. Además, de acuerdo con [27], los sistemas complejos están estrechamente relacionados con la teoría del caos, pues la interacción de múltiples elementos locales puede llevar al sistema completo a lo que llama "borde del caos", una región donde el sistema es lo suficientemente estable para mantener su estructura, pero se mantiene flexible para generar nuevos patrones.

**Definición 4.** *Un sistema complejo es una entidad compuesta de partes interconectadas, de tal forma que el comportamiento colectivo es más que la suma de sus componentes individuales.*

Para entender el comportamiento de un sistema es necesario conocer los elementos y sus interacciones. Es decir, se requiere conocer los estados, valores instantáneos de todos los elementos; y sus transiciones, es decir, los cambios dinámicos en los estados.

Los sistemas complejos se caracterizan principalmente por:

- **No linealidad**: No cumple el principio de superposición, es decir, el resultado no puede expresarse como la suma de las respuestas individuales a cada estímulo por separado. Resulta difícil predecir su evolución dinámica futura [26, 28].
- **Retroalimentación**: Las salidas pasadas influyen en las siguientes salidas. Este comportamiento hace alusión a la memoria del sistema [28].
- **Orden espontáneo**: Existe un orden emergente resultado de las interacciones locales no coordinadas, el cual no puede ser descrito mediante sus interacciones locales [28].

  **Aparición (*emergence*)**: Entidades, patrones, comportamientos y propiedades emergen de la interacción de entidades más pequeñas, las cuales no exhiben estos atributos [28].

  **Elementos múltiples**: Compuesto de múltiples elementos relativamente idénticos con interacciones locales [26].

## 2.4. Modelado de sistemas complejos

Resulta evidente que predecir el comportamiento de estos sistemas es una tarea complicada. Una aproximación es el uso de modelos científicos y matemáticos,

los cuales se construyen para desarrollar procesos de inferencia sobre aspectos observados en la vida real. El modelo de inferencia se realiza para entender el sistema real observado.

El proceso, de manera simplificada, se compone de tres fases, como se observa en la figura 2.2:

- Se abstrae el sistema real a un modelo, el cual puede trascender a otros sistemas reales.
- El modelo realiza un proceso de inferencia que busca averiguar las implicaciones derivadas de las premisas.
- Los resultados se utilizan para analizar, interpretar y aplicar a un sistema real. Esta aplicación no requiere ser práctica; es una relación entre las inferencias obtenidas con el modelo y el sistema real.

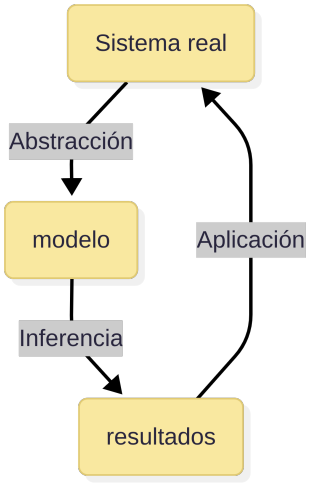



Figura 2.2: Representación gráfica del modelado científico. Este proceso busca relacionar eventos del mundo real utilizando un modelo que realiza una inferencia.

Sin embargo, estos modelos suelen carecer de las herramientas necesarias que les permiten deducir las implicaciones lógicas necesarias. Es aquí donde la computación permite agilizar este proceso realizando simulaciones.

Las computadoras permiten explorar y analizar modelos formales para resolver modelos matemáticos. Además, dado que un modelo computacional puede ser

definido mediante un modelo matemático, el rol de la computación es simplificar este proceso. Dicho esto, [29] define a la simulación de la siguiente manera:

**Definición 5.** *Simulación es un proceso computacional que permite imitar el comportamiento de un sistema mediante la utilización de un modelo matemático.*

Los resultados de la simulación provienen del uso de reglas algorítmicas. Por eso, es necesario ampliar el proceso de abstracción y crear un modelo formal que será utilizado por la computadora, como se muestra en la figura 2.3.

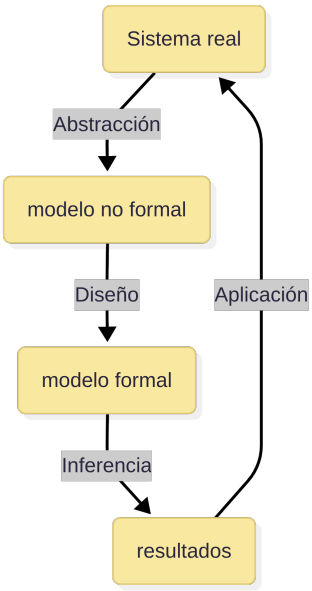



Figura 2.3: Representación gráfica del modelado computacional. Relaciona eventos del mundo real utilizando un modelo computacional que ayuda a agilizar el proceso de inferencia.

Se requiere de un experto en el sistema real que pueda abstraer el problema y un modelador que formalice dicha abstracción para ejecutarla en una computadora que infiera los resultados, mismos que utilizará nuevamente el experto en el sistema real para entablar la relación con el sistema real [30].

Para estudiar sistemas complejos hay varias herramientas de simulación. Una de ellas son los autómatas celulares [24, 27, 31], los cuales siguen la caracterización de los sistemas complejos: elementos idénticos con interacciones locales que pueden generar orden espontáneo, comportamientos emergentes e incluso retroalimentados.

# Capítulo 3

# Marco teórico: Autómatas celulares

La investigación de los sistemas complejos es un reto actual de la ciencia moderna. Estos sistemas, caracterizados por la interacción entre múltiples componentes locales, son capaces de generar comportamientos emergentes y, a veces, caóticos. Para entender esta complejidad necesitamos herramientas que puedan capturar la esencia de la autoorganización, y aquí los autómatas celulares surgen como un modelo computacional clave para simular fenómenos tanto naturales como artificiales.

En este capítulo se definen los fundamentos matemáticos y estructurales de un autómata celular. Este se entiende como un sistema dinámico discreto, que combina un conjunto de celdas interconectadas y que pueden producir patrones complejos a partir de reglas locales. Se mencionarán sus elementos esenciales, tales como:

- El vecindario: El subconjunto de celdas que afecta el cambio de una unidad.
- El estado de conjunto: Los valores finitos que puede tomar cada celda.
- Las condiciones de frontera son: El comportamiento del sistema en sus límites en el espacio.
- Las funciones de transición son: El algoritmo que rige la evolución del sistema.

Se revisan, asimismo, ejemplos que han sido de una importancia capital para el desarrollo de esta disciplina, tales como el Juego de la Vida de John Conway y los autómatas celulares elementales de Stephen Wolfram.

Finalmente, el capítulo amplía la definición clásica hacia terrenos más avanzados. Se presentan autómatas celulares que operan en espacios no euclidianos y se examinan extensiones recientes que incluyen estados continuos, reglas de probabilidad o actualizaciones asíncronas. Estas variaciones hacen que los AC continúen siendo, décadas después de ser concebidos por Von Neumann, una herramienta de vanguardia en campos tan diversos como la biología teórica, la criptografía y la física de partículas.

## 3.1. Autómata celular

Los autómatas celulares, concebidos por John von Neumann en 1948 [32], son una colección de células donde cada una de ellas puede adquirir un estado y se agrupan en una red. Además, estas células evolucionan en intervalos y cada una de ellas actualiza su estado en función del estado de sus células vecinas [33, 34, 35].

Siguiendo de manera general a [36, 37, 38], y considerando una red regular de células, un conjunto finito de estados y evoluciones en pasos discretos de tiempo, se define a un autómata celular como:

**Definición 6.** *Un autómata celular es una 4-tupla $(\mathcal{L}, \mathcal{S}, \mathcal{N}, \mathcal{R})$ donde:*

- $\mathcal{L} \subseteq \mathbb{Z}^D$ *es el espacio celular D-dimensional. El conjunto de células vive en el espacio* $\mathcal{L}$ *y cada célula* $\vec{i}$ *se define como un vector D-dimensional que representa su posición en el espacio.*
- $\mathcal{S}$ *es un conjunto de k estados. Cada célula* $\vec{i}$ *solo puede adquirir un valor de un conjunto finito de diferentes estados:*

$$\sigma_{\vec{i}}(t) \in \mathcal{S} \equiv \{s_0, s_1, s_2, \ldots, s_{k-1}\},\ \forall \vec{i} \in \mathcal{L}$$

  *Donde* $\vec{i} \in \mathcal{L}$ *es una célula en el espacio celular,* $\sigma_{\vec{i}}(t)$ *es el estado de la i-ésima célula en el tiempo t y* $\mathcal{S}$ *es un anillo conmutativo finito, como un intervalo de números enteros, que generalmente se define por los enteros módulo k (*$\mathbb{Z}_k$*).*
- $\mathcal{N} = (\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_m)$ *es el conjunto de los m vectores de desplazamiento que definen la vecindad de una célula en* $\mathcal{L}$*. Así, los vecinos de las células ubicadas en* $\vec{i} \in \mathcal{L}$ *se encuentran en las posiciones* $\vec{i} + \vec{v}, \forall \vec{v} \in \mathcal{N}$
- $\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$ *es la regla local del autómata:*

$$\sigma_{\vec{i}}(t+1) = \mathcal{R}(\{\sigma_{\vec{i}+\vec{v}_j}\}_{j=1}^{m})$$

*donde $\mathcal{R}$ es la función de transición local que asigna el nuevo estado de una célula a partir de los estados de sus m vecinos.*

## 3.2. Vecindad

Como ya se mencionó, la vecindad es el conjunto de vectores de desplazamiento, por lo que está estrechamente relacionado con la dimensionalidad del autómata celular. Por ejemplo, John von Neumann propuso una vecindad para $\mathbb{Z}^2$ de cinco células que actualmente se conoce como vecindad de von Neumann. Esta vecindad considera a la célula y a sus cuatro vecinas ortogonales más cercanas [39], dicho de otro modo:

$$\mathcal{N} = ((0,-1),(-1,0),(0,0),(1,0),(0,1))$$

La cual se puede representar como en la figura 3.1a.

Por otro lado, Moore añade los cuatro vecinos diagonales más cercanos a la vecindad de von Neumann, dando una vecindad de nueve elementos [40]. Esta vecindad, representada por la figura 3.1b, fue utilizada por John Conway en el "Juego de la Vida" [41].

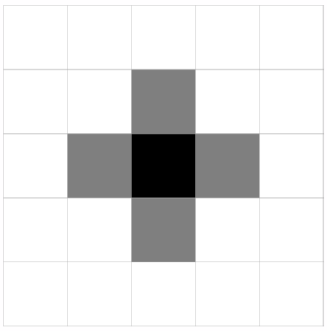
Otro ejemplo se encuentra en la figura 3.1c, la cual representa una vecindad de radio uno en un espacio unidimensional, es decir, todas aquellas células a una distancia menor o igual a uno, incluyéndose a su misma. Fue utilizada por Stephen Wolfram para definir los autómatas celulares elementales [42].

Las vecindades no solo son cuadradas; también pueden tener otras formas, como hexágonos o triángulos, que se muestran en las figuras 3.2a y 3.2b. Además, existen vecindades con comportamiento dinámico, como la de Margolus mostrada en la figura 3.2c, cuya configuración alterna entre dos disposiciones en cada paso evolutivo [43].

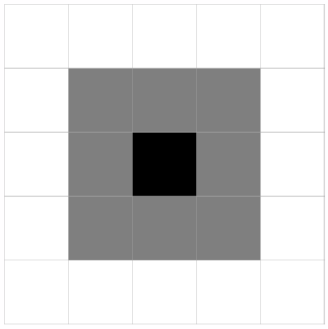
Finalmente, los vecinos no tienen que ser estrictamente adyacentes entre sí ni ser simétricos. Existen vecindades, como en la figura 3.3, donde algunas células no son contiguas a la central.

## 3.3. Estados

Cada célula adquiere un único estado $s$ en cada paso de tiempo, donde $s$ pertenece a un conjunto de estados $s \in \mathcal{S}$. Por ejemplo, el autómata celular de John von



(a) Vecindad de von Neumann

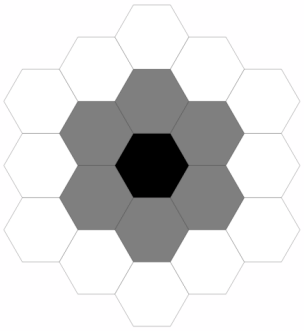


(b) Vecindad de Moore

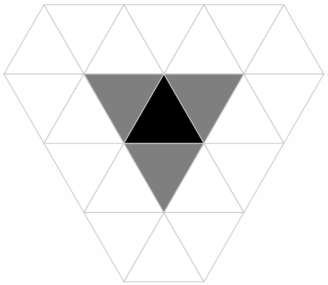


(c) Vecindad de radio uno en $\mathbb{Z}^1$

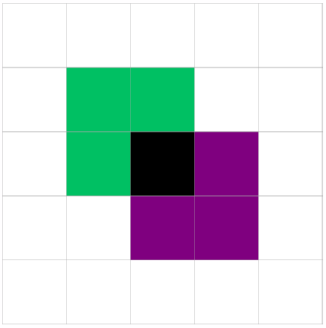
Figura 3.1: Las vecindades 3.1a y 3.1b son para CA bidimensionales; la 3.1c es para CA unidimensionales. La célula negra representa la célula actual y, junto con las grises, conforma su vecindad.



(a) Vecindad hexagonal



(b) Vecindad triangular



(c) Vecindad de Margolus

Figura 3.2: En 3.2a y 3.2b, la célula negra es la actual y las grises su vecindad. En 3.2c, la célula negra es la actual, mientras que las células verdes y moradas representan las vecindades activas en tiempos pares e impares.



Figura 3.3: Vecindad en $\mathbb{Z}^1$ con vecinos de adyacentes.

Neumann contempla 29 estados, aunque algunos autores han propuesto variantes reducidas a cuatro estados, conservando su capacidad de autoreproducirse [36].

Por otro lado, el juego de la vida se compone de dos estados que denominó "vivas" y "muertas" [41]. De igual forma, Stephen Wolfram utiliza dos estados en los autómatas celulares elementales [42].

Idealmente, el espacio celular $\mathcal{L}$ es infinito, lo que permitiría infinitas combinaciones de estados. Sin embargo, esto es inviable computacionalmente. Para esto, se requiere establecer una restricción de finitud, como lo marca [36].

Supóngase un espacio discreto unidimensional infinito. Idealmente, conocemos el estado de las infinitas células, pero computacionalmente no podemos almacenar ni procesar toda esa información. Por lo que se restringe a los valores que únicamente nos interesan.

A todas aquellas células que son de interés se les define con un estado activo, el cual puede ser cualquier $s \in \mathcal{S}$, por otro lado, las que no son de interés se les define con un estado inactivo $\sigma_q$:

**Definición 7.** *Sea $\mathcal{R}$ la regla local de un autómata celular. Un estado $\sigma_q \in \mathcal{S}$ es inactivo si $\mathcal{R}(\sigma_q, \sigma_q, \ldots, \sigma_q) = \sigma_q$.*

Es decir, si una célula y todos sus vecinos están en estado inactivo, la célula permanece inactiva tras aplicar $\mathcal{R}$. Cabe notar que todos los vecinos deben estar en dicho estado, por lo que, si alguno cambia a cualquier estado $s$, entonces se activa. Este evento ocurre cuando el espacio en donde evoluciona el autómata celular incrementa.

Para trabajar computacionalmente, [36] define las configuraciones finitas de la siguiente manera:

**Definición 8.** *Sea $\sigma_q \in \mathcal{S}$ un estado inactivo, $\sigma_p \neq \sigma_q \in \mathcal{S}$ un estado no inactivo y sea $\mathcal{L}$ un espacio celular infinito. Se considera que un autómata celular tiene configuración finita si solo un número finito de celdas están en estados activos $\sigma_p$.*

Si la configuración inicial es finita, la propiedad de finitud se preserva en cada paso de tiempo, como se observa en la tabla 3.1. Imaginemos un autómata celular en $\mathbb{Z}^1$ con una vecindad de radio 1 y una sola célula activa en la posición $n$ con estado $\sigma_p$, mientras que el resto de infinitas células tienen estado inactivo $\sigma_q$.

En el tiempo $t+1$ las células en $n-1$ y en $n+1$ tienen un vecino que no es inactivo, por lo que pasan a ser activas por la definición 7. Dado que este

| $t$ | $\vec{\sigma}(t)$ | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | | | | $\ldots$ | $q$ | $p$ | $q$ | $\ldots$ | | | |
| 1 | | | $\ldots$ | $q$ | $p$ | $p$ | $p$ | $q$ | $\ldots$ | | |
| 2 | | $\ldots$ | $q$ | $p$ | $p$ | $p$ | $p$ | $p$ | $q$ | $\ldots$ | |
| 3 | $\ldots$ | $q$ | $p$ | $p$ | $p$ | $p$ | $p$ | $p$ | $p$ | $q$ | $\ldots$ |

Tabla 3.1: Evolución temporal de un autómata celular unidimensional con una configuración inicial finita. Donde $p$ representa estados activos y $q$ estados inactivos. Los conjuntos $\vec{\sigma}(t)$ y $\vec{\sigma}(t+n)$ son finitos.

incremento está acotado por el tamaño de la vecindad, la configuración resultante sigue siendo finita. Por inducción, este proceso se repite para cualquier tiempo $t+m$; por lo tanto, mientras el tiempo de la evolución no tienda a infinito ($t \not\to \infty$) el autómata celular mantendrá una configuración finita, garantizando su viabilidad computacional.

Sea $\mathcal{C} = \mathcal{S}^{\mathcal{L}}$ el conjunto de todas las configuraciones. Definimos $\mathcal{C}_F \subset \mathcal{C}$ como el subconjunto de todas las configuraciones finitas. Esta restricción es necesaria para hacer computacionalmente viable la simulación de la evolución.

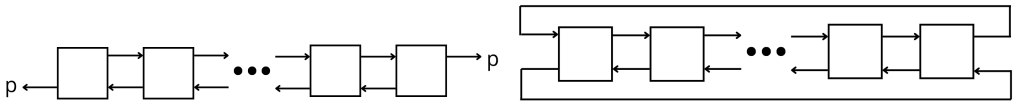
## 3.4. Condiciones de frontera

Idealmente, los autómatas celulares pueden evolucionar en espacios infinitos; sin embargo, se pueden establecer límites o fronteras, las cuales restringen el espacio celular $\mathcal{L}$ [36, 44].

**Definición 9.** *Un autómata celular es finito si el espacio celular $\mathcal{L} \subseteq \mathbb{Z}^D$ es finito; en caso contrario, se denomina infinito.*

Un autómata celular finito siempre tiene fronteras en los límites de su espacio. A la forma en como se comporta la frontera se le conoce como condición de frontera, la cual suele ser abierta o periódica [44].

- **Condición de frontera abierta**: Se les asigna un estado fijo a todos aquellos vecinos faltantes en las células de la frontera.
- **Condición de frontera periódica**: Los vecinos faltantes de las células en la frontera son otras células en la frontera. La figura 3.4b muestra cómo se conectan las células de la frontera en un espacio unidimensional.




(a) Condición de frontera abierta (b) Condición de frontera periódica

Figura 3.4: Condiciones de frontera con dos vecinos. La figura 3.4a asigna un estado no inactivo al vecino faltante. La figura 3.4b asigna como vecino el estado de la célula en la frontera opuesta, formando un anillo.

Un autómata infinito que tiene una configuración periódica equivale a un autómata finito con frontera periódica [36].

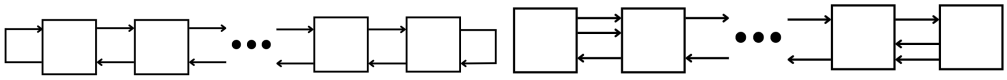
**Definición 10.** *Una configuración $\vec{\sigma} \in \mathcal{C}$ de un autómata celular $D$-dimensional se dice periódica si es invariante ante un conjunto de traslaciones linealmente independientes en $D$ direcciones. Es decir:*

$$\vec{\sigma} = T_i^{n_i}(\vec{\sigma}),\ \forall i \in \{1, 2, \ldots, D\}$$

*donde $T_i^{n_i}$ denota un desplazamiento de $n_i$ posiciones en la dimensión $i$.*

Definimos entonces a $\mathcal{C}_P \subset \mathcal{C}$ como el conjunto de todas las configuraciones periódicas. Estas configuraciones preservan la condición de frontera periódica en cada traslación [36].

La frontera no solo puede ser abierta o periódica, existen diferentes comportamientos como la reflectiva y la reflexiva. La frontera reflectiva asigna a los vecinos faltantes el mismo estado de la célula en la frontera (figura 3.5a), por lo que son vecinas de si mismas. La frontera reflexiva, en cambio, genera una doble relación de vecindad en los extremos (figura 3.5b), [36, 44].



(a) Condición de frontera reflectiva (b) Condición de frontera reflexiva

Figura 3.5: Condiciones de frontera con dos vecinos. La figura 3.5a muestra que las células en la frontera son vecinas entre sí, mientras que la figura 3.5b indica que las células que están al lado de las células de la frontera pueden tener múltiples relaciones de vecindad.

## 3.5. Regla local

Sea $\mathcal{S}$ el conjunto de estados y $m = |\mathcal{N}|$ el número de vecinos. La función de transición o regla local se define como

$$\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$$

donde $\mathcal{R}$ asigna un nuevo estado a cada célula en función de los estados de sus $m$ vecinos (incluyéndose a sí misma si aplica).

Para autómatas celulares unidimensionales, dado un rango arbitrario $r$, la regla puede ser escrita como:

$$\sigma_i(t+1) = \mathcal{R}(\sigma_{i-r}(t), \sigma_{i-r+1}(t), \ldots, \sigma_i(t), \ldots, \sigma_{i+r}(t)),\ \mathcal{R} : S^{2r+1} \to S$$

Donde el radio $r$ especifica el tamaño de la vecindad para cada célula (la vecindad es de tamaño $2r+1$ como se muestra en la figura 3.1c), la cual puede adquirir un valor $k$.

Un ejemplo de regla local es el juego de la vida [41]. En este caso, el conjunto de estados es binario $\mathcal{S} = \{0, 1\}$, donde 0 representa células muertas y 1 las vivas. La regla de transición realiza el siguiente mapeo:

$$\mathcal{R} : \{0, 1\}^9 \to \{0, 1\}$$

Ya que la vecindad de Moore en 2D involucra 8 vecinos más la propia célula, es decir, $m = 9$.

El juego de la vida sigue las siguientes tres reglas:

- **Nacimiento**: Una célula muerta en el tiempo $t$ nace en el tiempo $t+1$ si tiene exactamente tres vecinos vivos en el tiempo $t$.
- **Muerte**: Una célula viva en el tiempo $t$ muere en $t+1$ por:
  - Sobrepoblación; si hay más de tres vecinos vivos en $t$.
  - Soledad: si hay menos de dos vecinos vivos en $t$.
- **Supervivencia**: Una célula viva en el tiempo $t$ se mantiene viva en $t+1$ si tiene dos o tres vecinos vivos en $t$.

| 111 | 110 | 101 | 100 | 011 | 010 | 001 | 000 |
|---|---|---|---|---|---|---|---|
| ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ |
| $\{0,1\}$ | $\{0,1\}$ | $\{0,1\}$ | $\{0,1\}$ | $\{0,1\}$ | $\{0,1\}$ | $\{0,1\}$ | $\{0,1\}$ |

Tabla 3.2: Cada posible combinación de tres células puede adquirir uno de dos valores. Cuando la célula evaluada encaja en este patrón, su valor cambia al establecido por la regla. Este conjunto de 256 reglas se le conoce como autómata celular elemental.

Esta se conoce como regla semitotalística, ya que la transición depende de la suma de los valores en la vecindad, pero distingue entre diferentes posiciones, como el centro y los bordes [45].

Por otro lado, una regla es totalística si la transición depende únicamente de la suma de los estados en todos los vecinos, sin distinguir posiciones [46]. Para un autómata celular unidimensional con $|\mathcal{S}| = 3$ estados y radio $r = 1$, solo hay $(|\mathcal{S}| - 1)|\mathcal{N}| + 1 = (2 - 1) \cdot 3 + 1 = 4$ posibles valores de suma.

Finalmente, una regla local es arbitraria si distingue las posiciones de cada elemento en la vecindad. Cabe mencionar que las reglas arbitrarias contienen todas las posibles combinaciones de reglas, incluyendo las semitotalísticas, las cuales incluyen a su vez las totalísticas:

$$\mathcal{R}_{\text{totalísticas}} \subset \mathcal{R}_{\text{semitotalísticas}} \subset \mathcal{R}_{\text{arbitrarias}}$$

El objetivo de las reglas semitotalísticas y totalísticas será entonces simplificar ciertos comportamientos.

La configuración dada para el autómata celular elemental considera $k = 2$ (es decir, $\sigma_i \in \{0, 1\}$), un radio $r = 1$ ($|\mathcal{N}| = 3$) y la regla $\mathcal{R} : \{0, 1\}^3 \to \{0, 1\}$ que se define como:

$$\phi(\sigma_{i-1}(t), \sigma_i(t), \sigma_{i+1}(t))$$

Esta dinámica está dada por las posibles configuraciones que las tres células adyacentes pueden adquirir en el tiempo $t$. Podemos encontrar $k^{|\mathcal{N}|} = 2^3 = 8$ patrones posibles en la vecindad, número correspondiente a la combinatoria de dos estados en tres posiciones. Cada vez que encontremos un patrón, podemos asignarle un nuevo estado a $\sigma_i(t + 1)$ ($\{0, 1\}$), por lo que el número total de reglas asciende a $k^{k^{|\mathcal{N}|}} = 2^8 = 256$, como se muestra en la tabla 3.2.

Estas reglas se pueden representar de forma tabular y también como un número decimal. La tabla 3.3 muestra la asociación entre la forma tabular y la regla decimal.

| **Binario** | 111 | 110 | 101 | 100 | 011 | 010 | 001 | 000 | |
|---|---|---|---|---|---|---|---|---|---|
| **Decimal** | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | **regla** |
| **Valor** | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ | |
| | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| | | | | | ⋮ | | | | ⋮ |
| | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 30 |
| | | | | | ⋮ | | | | ⋮ |
| | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 110 |
| | | | | | ⋮ | | | | ⋮ |
| | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 255 |

Tabla 3.3: Los valores binarios representan las posibles configuraciones de estados que pueden aparecer en el espacio, cada uno asociado a un valor decimal, y el valor es la potencia asociada a esa combinación. Para convertir la regla tabular en la decimal, solo es necesario convertir el número binario.

Nótese que la regla decimal solo aplica cuando las condiciones son las mismas. Es decir, cuando el número de vecinos $|\mathcal{N}|$ y estados $|\mathcal{S}|$ son invariantes; a cada regla se le puede asociar un entero único, pero dos reglas que difieren en vecindad o estados pueden compartir el número decimal. Por ejemplo, considerando la regla 10 con $|\mathcal{N}| = 2$ y $|\mathcal{S}| = 2$ es 1010, la cual es completamente diferente cuando $|\mathcal{N}| = 3$ y $|S| = 2$, lo que da como resultado 000000101.

## 3.6. Ejemplos

### 3.6.1. John Von Neumann

El matemático y físico John von Neumann (1903-1957) fue uno de los pioneros en el estudio de los autómatas celulares, especialmente en lo que atañe a la reproducción automática y la universalidad computacional. En su obra póstuma, *Theory of Self-Reproducing Automata*, presentó un modelo de autómata celular bidimensional diseñado para construir máquinas autorreplicantes.

A continuación se presentan los principales elementos de su autómata celular, adaptados al formato de definición formal de este tipo de autómata:

- **Espacio celular:** Von Neumann definió un espacio bidimensional como una cuadrícula regular de celdas, idealmente infinita.
- **Estados:** En su versión original, cada célula podía adoptar uno de 29 estados posibles que permiten distinguir células en reposo, células en transmisión de "señal", células constructoras, etc.
- **Vecindad:** Utiliza la vecindad que más tarde se denominaría "vecindad de von Neumann" [47].
- **Función de transición:** Todas las células actualizan su estado de forma síncrona, de acuerdo con una regla común que depende del estado actual de la propia célula y de los estados de sus vecinos. En el modelo de von Neumann, esta regla está diseñada de modo que permita que ciertas configuraciones de células funcionen como un constructor universal capaz de autorreplicarse.

Su objetivo era demostrar de manera formal que se puede crear un autómata celular que se reproduzca sin fin y que, además, pueda hacer construcción universal dentro de ese sistema celular [39, 48].

### 3.6.2. El juego de la vida

El juego de la vida es un autómata celular propuesto por el matemático John Horton Conway en 1970. Consiste en una cuadrícula de celdas que pueden estar en uno de dos estados posibles: vivas o muertas. A lo largo del tiempo, las celdas evolucionan de acuerdo con un conjunto de reglas locales que determinan su supervivencia, muerte o nacimiento, en función del número de vecinos vivos que posean [49].

Siguiendo la definición formal de un autómata celular, el *Juego de la Vida* se caracteriza por los siguientes elementos:

- **Espacio celular**: Se define sobre una cuadrícula bidimensional regular, teóricamente infinita.
- **Estados**: Cada célula puede encontrarse en un estado $\{\text{muerto}, \text{vivo}\}$, representados comúnmente como $\{0, 1\}$.
- **Vecindad**: Se utiliza una vecindad de Moore semitotalística, la cual incluye las ocho celdas adyacentes a la célula central.

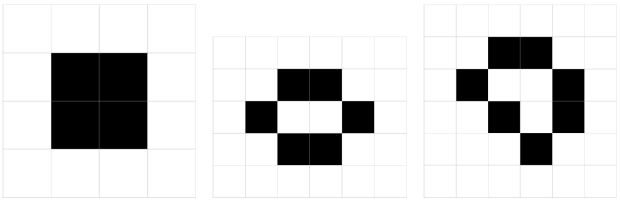
- **Función de transición**: El estado futuro de cada célula depende de su estado actual y del número de vecinos vivos, siguiendo las reglas:
  - Una célula viva sobrevive cuando tiene dos o tres vecinos vivos.
  - Una célula viva con más de tres vecinos vivos muere por sobrepoblación.
  - Una célula viva que tiene menos de dos vecinos vivos muere por soledad.
  - Una célula muerta con exactamente tres vecinos vivos renace.

Estas reglas pueden modificarse para crear variantes del autómata, cambiando las condiciones de nacimiento o supervivencia. En [41] se analizan distintas funciones de transición y configuraciones espaciales.

Una notación común es *Birth/Survival*, que especifica los números de vecinos vivos requeridos para cada caso. El juego de la vida tradicional corresponde a la configuración $B3/S23$: una célula muerta nace con tres vecinos vivos, y una viva sobrevive si tiene dos o tres vecinos vivos [41].

Dentro de este sistema emergen patrones característicos, que se agrupan en tres categorías principales:

- **Naturaleza muerta**: Estructuras estáticas que permanecen invariantes en el tiempo.



(a) *Block* (b) *Beehive* (c) *Loaf*

Figura 3.6: Ejemplos de configuraciones de naturaleza muerta.

- **Osciladores**: Patrones que repiten su configuración después de un número finito de iteraciones, manteniendo su posición en la cuadrícula.

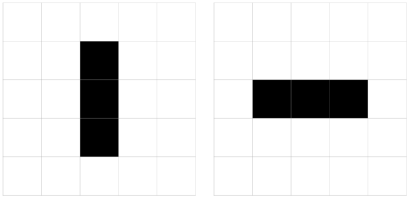


Figura 3.7: *Blinker*.

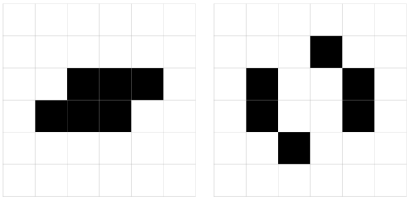


Figura 3.8: *Toad*.

- **Naves espaciales**: Son osciladores que, además de repetir su patrón, se desplazan por el espacio celular.

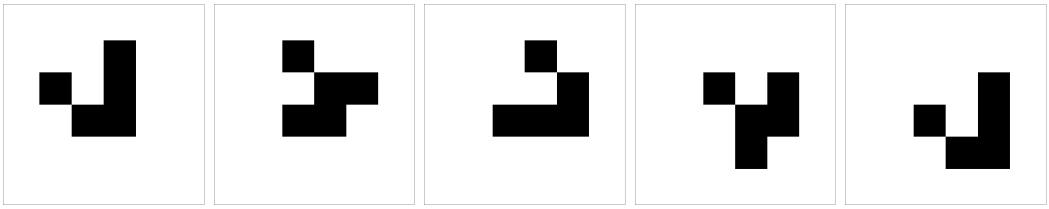


Figura 3.9: *Glider*.

Paul Rendell [50] demostró que, a partir de una configuración inicial apropiada, el *Juego de la Vida* es Turing completo, es decir, tiene la capacidad de realizar cualquier cálculo computable mediante la simulación de una máquina de Turing. La figura 3.10 muestra la configuración inicial del autómata celular diseñado por Paul Rendell.

Debido a su naturaleza semitotalística, la vecindad puede dividirse en dos conjuntos, y con ello determinar el número de posibles estados que puede adoptar cada uno:

Así, puede definirse un vector de 18 elementos que representa la función de transición, como se muestra en la tabla 3.4.

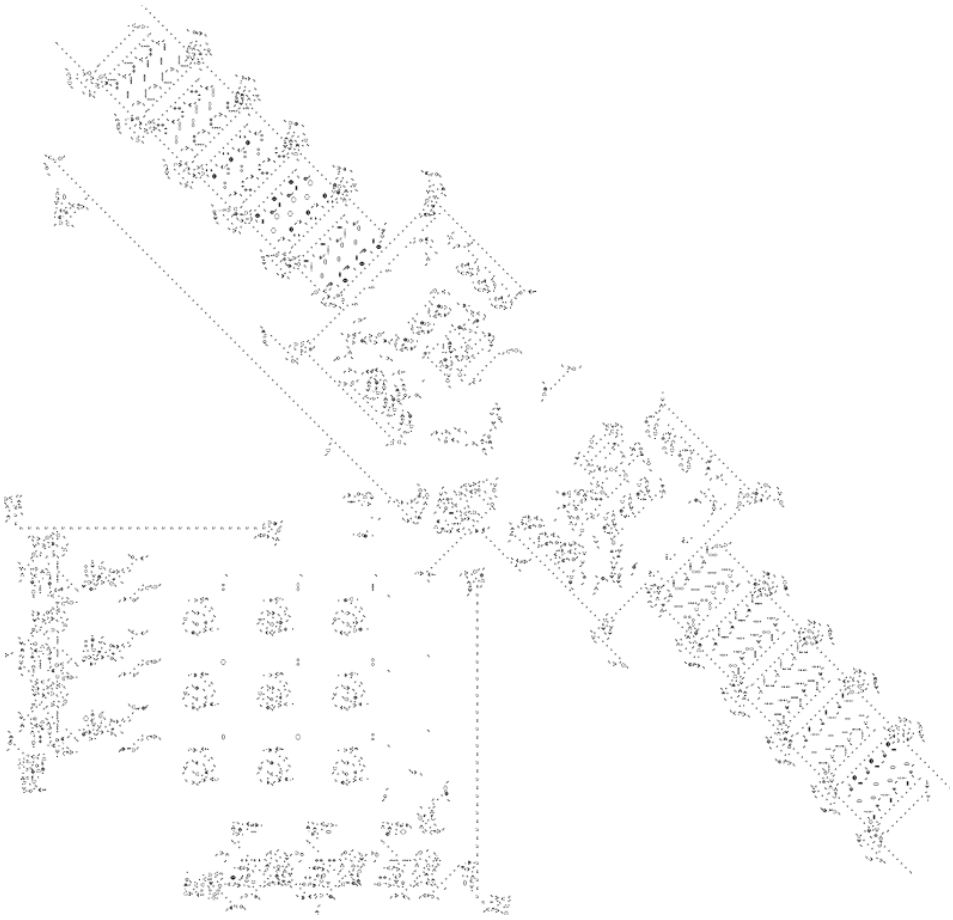


Figura 3.10: Máquina de Turing implementada en el *Juego de la Vida* [51].

| **Vecindad** | **Estados** | **Total** |
|---|---|---|
| Frontera | $\{0,1,2,3,4,5,6,7,8\}$ | 9 |
| Centro | $\{0,1\}$ | 2 |

| Frontera | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Centro | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| $B3/S23$ | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |

Tabla 3.4: Representación binaria de la regla $B3/S23$.

De manera general, el tamaño del vector que define una regla semitotalística puede calcularse mediante la ecuación 3.1. En este caso, dicho vector tiene tamaño 18, por lo que existen $|\mathcal{S}|^{18} = 2^{18} = 262,144$ posibles reglas.

$$\prod_{i=0}^{n}(|\mathcal{S}|-1)(|n|)+1, \quad \forall n \in \mathcal{N} \tag{3.1}$$

### 3.6.3. Autómata celular elemental

Stephen Wolfram propuso una versión más simple de los autómatas celulares, llamada autómata celular elemental. En esta versión, se disminuye la dimensionalidad del espacio celular y, por lo tanto, se reduce el tamaño de la vecindad. A continuación se detallan sus características según la definición formal de autómata celular:

- **Espacio celular**: Está formado por una cadena unidimensional de celdas dispuestas linealmente.
- **Estados**: Cada célula puede encontrarse en uno de dos estados posibles, $\{0, 1\}$.
- **Vecindad**: Se define mediante un radio $r$ que determina cuántas celdas adyacentes influyen en la evolución de cada célula. En el caso del autómata celular elemental, el radio es $r = 1$, por lo que la vecindad está compuesta por la célula central y un vecino a cada lado.
- **Función de transición**: Wolfram no propuso una única regla de evolución, sino el conjunto completo de funciones de transición posibles bajo estas condiciones.

Dado que cada célula considera tres posiciones (la propia y sus dos vecinas) y cada una puede tener dos posibles estados, existen $2^3 = 8$ combinaciones de vecindad. Por tanto, el número total de reglas posibles es $2^8 = 256$.

Wolfram asignó a cada una de estas funciones un número entero entre 0 y 255, conocido como número de regla, que identifica de forma unívoca el comportamiento del autómata. Además, propuso una clasificación de las reglas en cuatro categorías, según los patrones que emergen durante su evolución temporal (estas clases se detallan en la siguiente sección).

A continuación, se presentan tres reglas representativas por su relevancia teórica y sus propiedades computacionales.

- **Regla 30**: La Regla 30 pertenece a la clase de comportamiento caótico. A partir de una configuración inicial simple, genera patrones altamente complejos y aparentemente aleatorios [52] como se muestra en las figuras 3.11.

  Esta regla plantea tres interrogantes fundamentales sobre la naturaleza del caos y la computabilidad:

  1. ¿La columna central permanece no periódica a lo largo del tiempo?
  2. ¿Existe equilibrio estadístico entre los estados vivos y muertos?
  3. ¿Es posible calcular el estado de la célula central en el paso $n$ en tiempo $O(n)$?

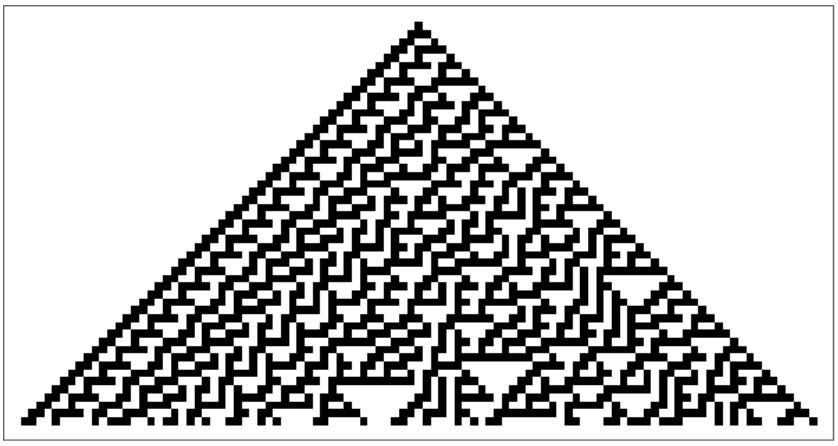
- **Regla 110**: La Regla 110 exhibe un comportamiento mixto: combina regiones periódicas y caóticas que interactúan entre sí de manera compleja. A partir de una única célula activa, genera estructuras móviles y patrones que pueden coexistir en el tiempo [53].

  Lo más notable de esta regla es su universalidad computacional. Wolfram y Matthew Cook demostraron que la Regla 110 es Turing completa, es decir, capaz de simular cualquier máquina de Turing y, por tanto, realizar cualquier cómputo posible en un sistema discreto determinista [53]. Su evolución se muestra en las figuras 3.12.

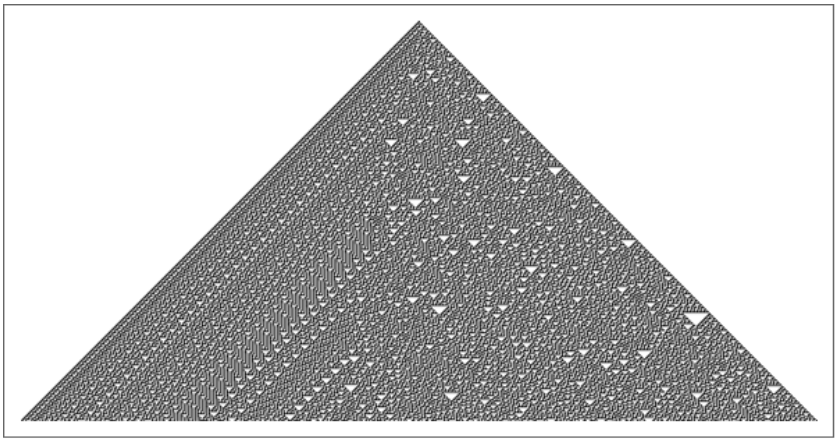
- **Regla 90**: La Regla 90 se distingue por generar estructuras fractales. Iniciando desde una única célula activa, produce el conocido triángulo de Sierpinski como se muestra en la figura 3.13, el cual es un patrón autosimilar que aparece en múltiples sistemas naturales y matemáticos [54].

## 3.7. Clasificación

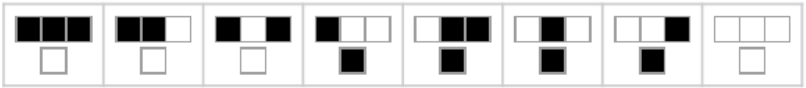
Los autómatas celulares elementales fueron utilizados por diferentes autores por representar la máxima simplicidad y mínima configuración que un autómata puede adquirir y que tenga un amplio rango de comportamientos. Múltiples autores han intentado definir una clasificación; a continuación se muestran algunas clasificaciones propuestas por diferentes autores.



(a) 50 evoluciones.

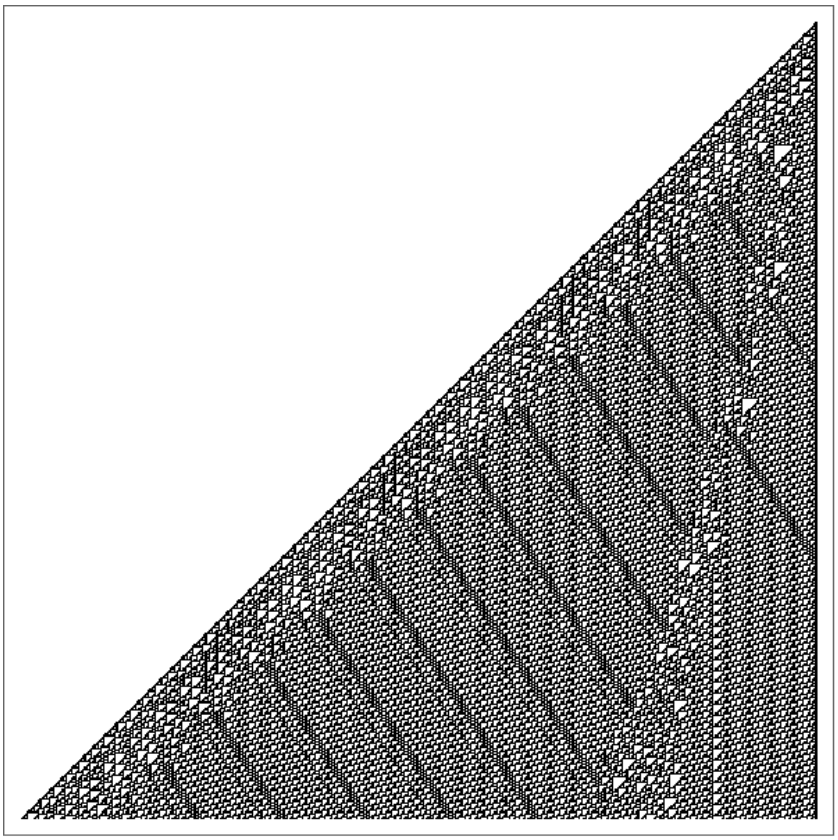


(b) 350 evoluciones.

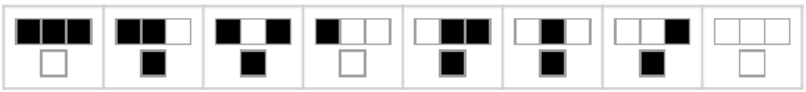


(c) Función de transición

Figura 3.11: Evolución de la regla 30

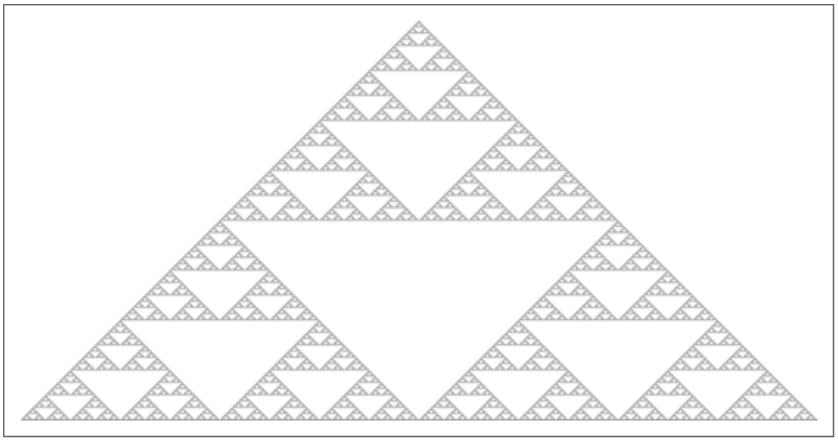


(a) 500 evoluciones.

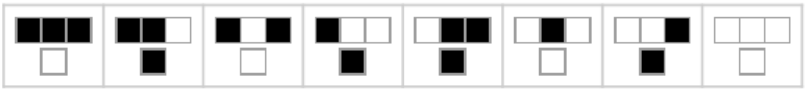


(b) Función de transición

Figura 3.12: Evolución de la regla 110



(a) 1025 evoluciones.



(b) Función de transición

Figura 3.13: Evolución de la regla 90

Stephen Wolfram realiza una clasificación basada en el comportamiento que exhiben ciertas reglas y que es inherente a la configuración inicial. Es por esto que define las siguientes clases de comportamiento, ilustradas en la figura 3.14:

WI Evolucionan a un estado uniforme.

WII Evolucionan a estados cíclicos aislados.

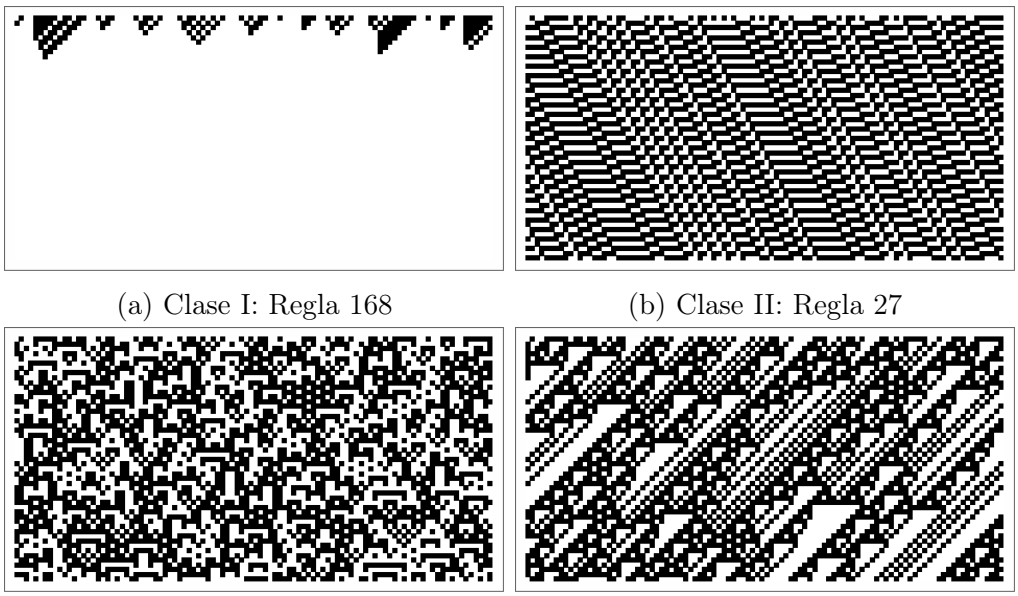
WIII Evolucionan en comportamientos caóticos.

WIV Evolucionan en estados complejos aislados.

Sin embargo, existen reglas bajo diferentes configuraciones que pueden pertenecer a múltiples clases o, en el peor de los casos, no pertenecen a ninguna de ellas [55].

Li y Packard ofrecen otra clasificación que considera seis clases para categorizar el comportamiento dinámico de los autómatas celulares [56]:

LPI La regla evoluciona a un estado homogéneo donde todas las células adquieren el mismo valor.



(a) Clase I: Regla 168 (b) Clase II: Regla 27

(c) Clase III: Regla 105 (d) Clase IV: Regla 106

Figura 3.14: Clasificación de Wolfram. Las figuras evolucionan 50 pasos con la misma configuración inicial y con condición de frontera cíclica.

LPII Evoluciona hasta una configuración estática no necesariamente homogénea.

LPIII Reglas con un periodo igual a dos.

LPIV Reglas con un periodo de $L$.

LPV Evolucionan a estructuras caóticas.

LPVI Conduce a estructuras largas y estructuras complejas.

Por otro lado, Culick y Yu [56, 57] proponen una clasificación que sigue las siguientes limitaciones:

- Esta clasificación es para autómatas celulares no totalísticos.
- Cada clase es un subconjunto de la siguiente clase.

CY I Un autómata celular es de clase I si existe un estado estable $s \in \mathcal{S}$ tal que todas las configuraciones finitas evolucionan a una configuración homogénea de $s$.

Es decir, dada cualquier configuración inicial en un autómata celular finito, todas las células adquieren el estado $s$ tras cierto número de pasos evolutivos. Esto se aplica a todas las finitas configuraciones iniciales.

CY II Un autómata celular es de clase II si existe un estado estable $s \in \mathcal{S}$ tal que se repite a $n$ pasos evolutivos.

Esto implica que la evolución se repite cada $n$ número de pasos ($\sigma(t) = \sigma(t+n)$). Resulta evidente que la clase I está contenida en la clase II, haciendo que $n = 1$, lo que implica un periodo de uno.

Una clasificación es $s$-finita si solo difiere del estado estable en una región finita.

CY III Un autómata celular es de clase III si existe un estado estable $s \in \mathcal{S}$ tal que para cualquier par de configuraciones $s$-finitas $C_1$ y $C_2$, es decidible si $C_1$ evoluciona a $C_2$.

Es decir, si podemos determinar en tiempo finito si el CA evoluciona de $C_1$ a $C_2$, entonces pertenece a clase III. En caso contrario, pertenece a la clase IV, la cual contiene a todos los autómatas celulares [57].

CY IV Todos los autómatas celulares pertenecen a la clase IV.

## 3.8. Extensiones y cambios de la definición

Otra clasificación considera extensiones o modificaciones en la definición que alteran el comportamiento del autómata celular. A continuación, se describen algunas de estas extensiones para el modelado de sistemas complejos.

- **Autómatas celulares en grafos**: Amplían la idea de la estructura de red clásica al permitir que las vecindades no sean iguales: cada célula puede tener un número diferente de vecinos. Esta flexibilidad los hace adecuados para modelar fenómenos sobre grafos, como redes sociales o flujos en infraestructura. Sin embargo, diseñar manualmente las reglas de transición en estos entornos es una tarea compleja, lo que ha motivado propuestas como los autómatas celulares neuronales en grafos, que permiten aprender dichas reglas automáticamente [58].
- **Autómatas celulares probabilísticos**: Estos extienden el modelo clásico al incorporar reglas de transición estocásticas. En vez de una evolución que

sigue un camino fijo, los estados futuros dependen de probabilidades que se ven afectadas por el entorno. Esto ayuda a entender la incertidumbre, el ruido y los comportamientos que surgen de manera inesperada. Son especialmente útiles para simular dinámicas como propagación de epidemias o sistemas sociales, donde los mecanismos de transición no siempre están completamente definidos [59].

- **Autómatas celulares no homogéneos**: Permiten que diferentes células tengan diferentes funciones de transición. Para esto, se utilizan dos planos llamados "plano de datos", el cual contiene la evolución del sistema; y "plano de control", el cual define las reglas de transición para cada célula. Esto permite estudiar el comportamiento y la interacción entre diferentes reglas que comparten espacio [60].
- **Autómatas celulares continuos**: Son autómatas celulares que pueden ser continuos en los estados, en el tiempo y en el espacio [42]:
  - Los estados continuos adquieren valores reales y no solo enteros.
  - El tiempo continuo se define mediante una función que determina el intervalo de espera para actualizar la célula en función de su estado actual.
  - El espacio continuo se obtiene al asignar diferentes pesos a las células dentro de la vecindad [61].

Estas variantes no son exclusivas y además han ampliado el uso de los autómatas celulares. Sin embargo, un problema que se repite es definir correctamente las reglas de transición para lograr un fenómeno específico. Este desafío ha llevado a la creación de autómatas celulares neuronales. En estos, la función de transición no se define de manera clara, sino que se aprende a través de una red neuronal artificial. Pero antes de adentrarse en la definición, es necesario comprender qué son las redes neuronales.

# Capítulo 4

# Marco teórico: Inteligencia artificial y redes neuronales

Las redes neuronales artificiales son una herramienta de inteligencia artificial inspirada en el pensamiento humano, más específicamente, en cómo un conjunto de neuronas interconectadas comparten información para realizar inferencias.

Las redes neuronales tienen su origen en 1943, donde McCulloch y Pitts propusieron un modelo que establecería las bases del perceptrón. Posteriormente, en 1957, Rosenblatt formalizó su implementación computacional, pero no fue hasta los años ochenta donde tuvo un impacto en su implementación gracias al surgimiento de la retropropagación y de las redes recurrentes [62].

Las redes neuronales artificiales son un conjunto de neuronas o módulos computacionales interconectados que realizan cálculos y transmiten información para llevar a cabo una operación, imitando el comportamiento del cerebro.

Sin embargo, para entender este modelo, es necesario tener los fundamentos de inteligencia artificial y la clasificación de aprendizaje supervisado. Es por esto que en este capítulo, se describen términos como inteligencia artificial, aprendizaje supervisado, la composición, estructura y comportamiento de las redes neuronales y algunos tipos de ellas.

## 4.1. Inteligencia artificial

La inteligencia artificial fue definida por primera vez por John McCarthy en 1955 como *la ciencia e ingeniería de hacer máquinas inteligentes* [63]. Es un término que ha ido evolucionando a lo largo de los años y con diferentes interpretaciones. Para el objetivo de esta investigación y siguiendo a [63, 64, 65], se sigue la siguiente definición:

**Definición 11.** *La inteligencia artificial es la ciencia que utiliza modelos computacionales para crear sistemas capaces de realizar tareas que, si fueran hechas por humanos, requerirían inteligencia.*

Una de las ramas de la inteligencia artificial es el aprendizaje automático, el cual se centra en desarrollar sistemas con la capacidad de aprender en función de los datos [66]. Esto permite reconocer patrones y realizar predicciones a partir de grandes cantidades de datos [67].

Por lo que, con base en [66, 68] podemos definir al aprendizaje automático como:

**Definición 12.** *El aprendizaje automático es la disciplina que permite a las computadoras mejorar su desempeño en una tarea mediante la experiencia con datos, sin necesidad de instrucciones explícitas.*

Esta tecnología funciona trabajando a través del entrenamiento de algoritmos en conjuntos de datos para conseguir un resultado esperado. Es el proceso de optimizar el modelo para que pueda predecir la respuesta correcta. A mayor calidad en los datos de entrenamiento, mayor precisión se tendrá en el modelo [67].

El entrenamiento se refiere al proceso de enseñarle a un modelo de inteligencia artificial a realizar una o varias tareas mediante la exposición a grandes cantidades de datos [69]. Este proceso facilita a los humanos el proceso de análisis de extensos volúmenes, especialmente en la era actual, donde se generan más datos que nunca en la historia [70].

Dentro del aprendizaje automático se definen tres métodos en cómo una computadora puede aprender. Estos métodos son aprendizaje supervisado, no supervisado y por refuerzo.

- El aprendizaje supervisado se caracteriza por el uso de conjuntos de datos etiquetados para entrenar algoritmos que clasifiquen datos o predigan resultados con precisión. Durante el proceso de entrenamiento, el modelo recibe

entradas junto con las salidas correctas, lo que le permite al modelo ajustar sus parámetros hasta que sea capaz de realizar predicciones precisas sobre nuevos datos [68].

- El aprendizaje no supervisado, a diferencia del supervisado, no necesita de un conjunto de datos etiquetados generados por el ser humano. Este tipo de aprendizaje puede descubrir patrones y estadísticas sin ninguna orientación o instrucción explícita [71].
- El aprendizaje por refuerzo busca imitar la forma de aprendizaje humana de prueba y error. Este tipo de aprendizaje permite a las máquinas aprender a tomar decisiones y mejorar sus resultados con base en los errores cometidos [72].

IBM [68] menciona algunos de los algoritmos más comunes de aprendizaje automático:

1. **Regresión lineal:** Predice valores numéricos, con base en una relación lineal entre diferentes valores.
2. **Agrupación en clústeres:** Identifican patrones en los datos y los agrupan en centroides.
3. **Árboles de decisión:** Utilizan una secuencia ramificada de decisiones vinculadas que se representan con un diagrama de árbol.
4. **Redes neuronales:** Simulan el funcionamiento del cerebro humano, con un gran número de nodos de procesamiento conectados entre sí, llamados neuronas.

Las redes neuronales presentan ventajas que pueden llegar a agilizar y optimizar recursos de las organizaciones que lo implementan [73]: mejora la productividad, reduce el error humano, mejora la disponibilidad de servicios, reduce tareas repetitivas, descubre patrones ocultos a simple vista, entre otros.

Para comprender cómo las redes neuronales logran estas capacidades, es necesario explorar primero los fundamentos que inspiraron el desarrollo. En particular su diseño es una abstracción de las neuronas biológicas, cuya estructura y funcionamiento ofrecen una analogía para el procesamiento de la información.

## 4.2. Neurona biológica

Las neuronas son células especializadas del sistema nervioso cuyo objetivo es recibir, procesar y transmitir información por medio de señales químicas y eléctricas [74, 75].

Cada neurona se compone (principalmente) de tres elementos, como se muestra en la figura 4.1:

- Las dendritas, que reciben señales provenientes de otras neuronas.
- El núcleo o cuerpo, que integra las señales provenientes de las dendritas y produce una nueva señal eléctrica.
- El axón, el cual conduce la señal producida por el cuerpo hacia otras neuronas haciendo sinapsis, es decir, transmite la señal de una célula a otra.

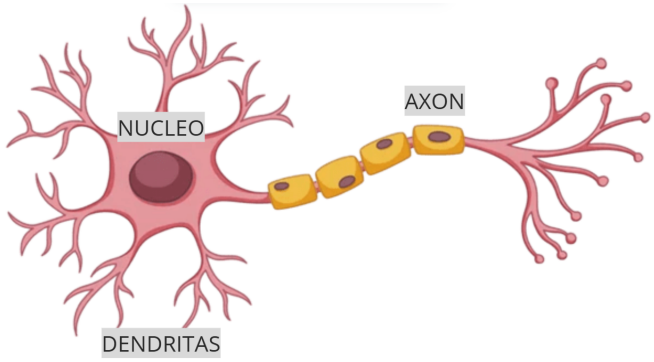



Figura 4.1: Representación de una neurona [76]. Se encuentran señaladas las dendritas, el núcleo y el axón, encargado de recibir, procesar y transmitir la información, respectivamente.

Las redes neuronales naturales son un sistema de neuronas que permite el procesamiento cerebral de la información; son altamente complejas, no lineales y con procesamiento en paralelo.

## 4.3. Neurona artificial

La neurona artificial se considera como una unidad de procesamiento de información [74]. Sigue la misma estructura del modelo biológico con algunas conside-

raciones adicionales, como lo marcan [74, 77, 78]:

- Las dendritas equivalen al conjunto de datos de entrada. Sin embargo, no toda la información tiene la misma relevancia; para esto se le asigna a cada entrada un peso sináptico.
- El cuerpo se sustituye por dos funciones: (1) una función de agregación que combina todos los datos de entrada (generalmente es una combinación lineal entre las entradas y los pesos sinápticos) y (2) una función de activación que determina si la neurona está activa y en qué grado.

La cual se suele representar como en la figura 4.2.

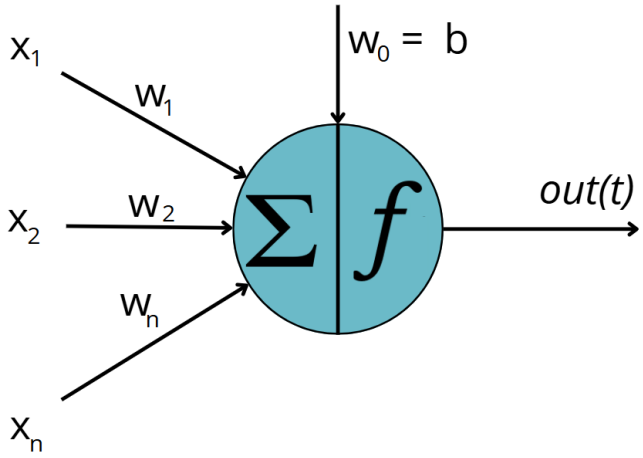



Figura 4.2: Representación gráfica de una neurona artificial. Contiene las entradas $(x_1, x_2, \ldots, x_n)$, los pesos sinápticos $(w_1, w_2, \ldots, w_n)$, el sesgo ($w_0$ o $b$), la función de agregación ($\Sigma$), la función de activación ($f$) y la salida.

**Definición 13.** *Una neurona artificial es una función* $f : \mathbb{R}^n \to \mathbb{R}$ *dada por:*

$$f(x) = \phi(\omega(X, W, b))$$

*donde:*

- $X = \{x_1, x_2, \ldots, x_n\} \in \mathbb{R}^n$ *es el conjunto de datos que recibe cada neurona.*
- $W = \{w_1, w_2, \ldots, w_n\} \in \mathbb{R}^n$ *es el conjunto de pesos sinápticos que indican la importancia de cada entrada.*

- *$b$ o $w_0 \in \mathbb{R}$ es un sesgo que desplaza la función de activación y controla el umbral de aceptación.*
- *$\omega(X, W, b)$ es una función de agregación $\omega : \mathbb{R}^n \to \mathbb{R}$ que combina las entradas. Usualmente se define como una combinación lineal:*

$$\omega(X, W, b) = \sum_{i=1}^{n} w_i x_i + b$$

- *$\phi(\omega(X, W, b))$ es la función de activación $\phi : \mathbb{R} \to \mathbb{R}$ que transforma $\omega(X, W, b)$ en una salida activada.*

Además de la combinación lineal, la función de agregación puede adoptar otras formas, como el máximo, el mínimo o combinaciones lógicas (AND, OR) en redes con entradas binarias. No obstante, la suma ponderada continúa siendo la más utilizada en aplicaciones prácticas [62].

La función de activación es la que determina el valor de salida de cada neurona. Este valor, normalmente, se encuentra acotado dentro de un rango específico (como $[0, 1]$ o $[-1, 1]$) para normalizar el comportamiento de la red [79]. Además, su naturaleza no lineal es fundamental para que el modelo pueda aproximar funciones complejas y resolver problemas que no son linealmente separables.

## 4.4. Redes neuronales

Una neurona únicamente puede realizar una operación. Para aumentar la capacidad de entender patrones más complejos, es necesario añadir múltiples neuronas interconectadas, lo que se conoce como red neuronal artificial [80]. Esta puede definirse como un grafo dirigido [81].

**Definición 14.** *Una red neuronal artificial es un grafo dirigido $G = (V, E)$ donde cada nodo $v \in V$ representa una neurona y cada arista $(u, v) \in E$ transporta la salida $y_u$ desde $u$ hasta $v$ ponderada por el peso $w_{uv}$.*

*Existe un conjunto de nodos de entrada $X \subseteq V$ que reciben los datos iniciales y un conjunto de nodos de salida $T \subseteq V$ cuya activación final representa la salida de la red.*

Aunque la topología de la red puede ser arbitraria, en la práctica las redes se organizan en tres conjuntos de capas. Esto se muestra en la figura 4.3, las cuales

son (1) una de entrada que recibe los datos, (2) una o varias ocultas y (3) una de salida que devuelve el resultado. Adicionalmente, suelen diseñarse redes neuronales completamente conectadas, es decir, todas las neuronas tienen una relación con todas las de la siguiente capa, pero no es una restricción [82].

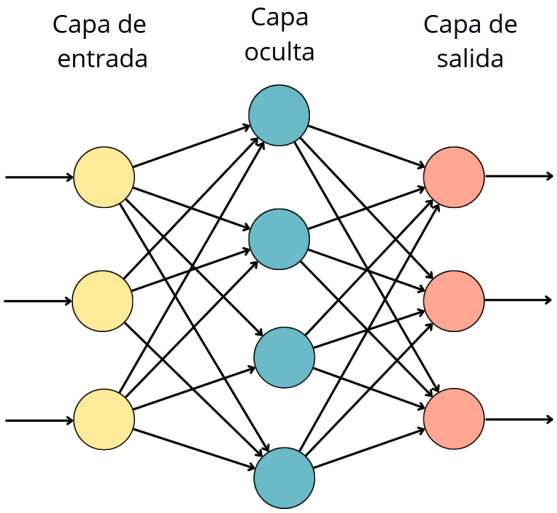



Figura 4.3: Red neuronal de tres capas: tres entradas, una capa oculta y tres salidas.

La red en su conjunto implementa una función parametrizada.

$$F_\theta : \mathbb{R}^n \to \mathbb{R}^m$$

donde $n$ es el número de datos de entrada, $m$ el número de datos de salida y $\theta$ el conjunto de parámetros entrenables. El entrenamiento de una red neuronal consiste en ajustar $\theta$.

Por lo que se puede concluir con la definición de [62], la cual dice que

**Definición 15.** *Una red neuronal es una función* $F_\theta : \mathbb{R}^n \to \mathbb{R}^m$ *donde* $\theta = \{w_{uv}, b_v : (u, v) \in E, v \in V\}$ *es el conjunto de parámetros entrenables.*

### 4.4.1. Funciones de activación

La capacidad de $F_\theta$ para modelar relaciones no lineales depende de las funciones de activación aplicadas en cada nodo $v$. Sin la función de activación, la red neuronal podría definirse únicamente como una función lineal.

En esta sección se presentan las funciones de activación más comunes utilizadas en redes neuronales. Para cada función se muestra su definición matemática, una breve descripción de su comportamiento práctico y una figura ilustrativa.

#### 4.4.1.1. Binary step (escalón binario)

La función escalón binario se define por partes:

$$f(x) = \begin{cases} 1 & \text{si } x \geq 0, \\ 0 & \text{si } x < 0. \end{cases}$$

Produce salidas discretas (0/1) y fue utilizada en perceptrones tempranos como función de decisión. No es diferenciable en el punto de umbral y su derivada es nula en casi todas partes, por lo que no es adecuada para entrenamiento mediante gradiente (backpropagation). Aun así, se usa conceptualmente para tomar decisiones discretas o como capa de postprocesado/umbral en modelos ya entrenados.

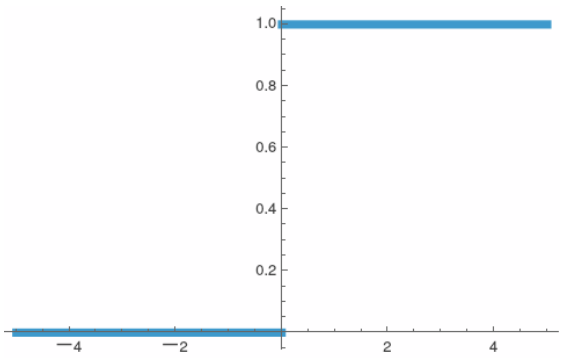



Figura 4.4: Función de activación escalón binario.

#### 4.4.1.2. Lineal

La función lineal (o identidad) se define como:

$$f(x) = ax + b \quad \vee \quad f(x) = x.$$

Produce una salida proporcional a la entrada y mantiene la linealidad en la red. Se utiliza principalmente en la capa de salida para tareas de regresión donde se requiere predecir valores continuos. No es adecuada para capas ocultas cuando se desea que la red aprenda transformaciones no lineales.

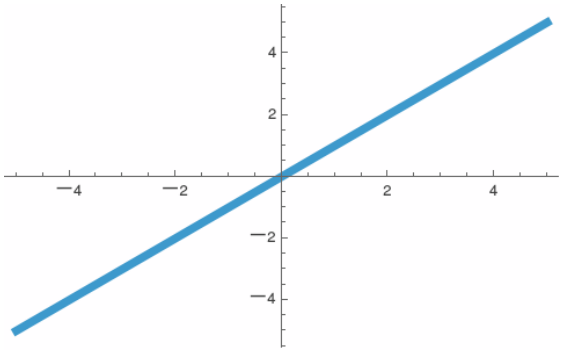



Figura 4.5: Función de activación lineal.

#### 4.4.1.3. Sigmoide

La función sigmoide logística se define como:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Tiene la propiedad de:

$$\sigma'(x) = \sigma(x)\,(1 - \sigma(x)).$$

Comprime la entrada en el intervalo $(0, 1)$, por lo que es apropiada para modelar probabilidades y se ha usado históricamente en la salida de clasificadores binarios. Sufre saturación cuando los valores son grandes, lo que provoca gradientes pequeños (esto se llama el problema del gradiente desvanecido) y una salida que no está centrada (es positiva). Por eso, en las capas ocultas modernas, a menudo se reemplaza por ReLU o tanh.

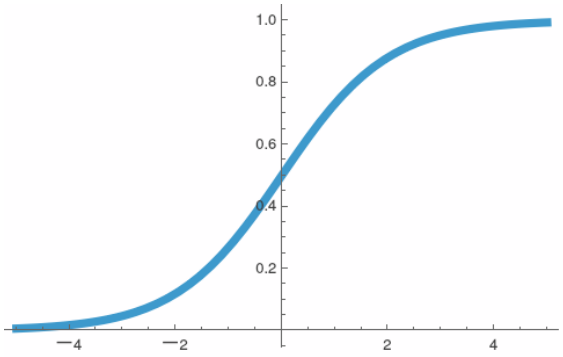



Figura 4.6: Función de activación sigmoide.

#### 4.4.1.4. tanh (hiperbólica)

La tangente hiperbólica se define como:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Su rango es $(-1, 1)$ y su derivada puede expresarse en función de sí misma.

Es una versión centrada en cero de la sigmoide, lo que facilita el aprendizaje (gradientes con medias cercanas a cero). Se usa en capas ocultas cuando se necesita una salida limitada y centrada. Era una opción común en problemas secuenciales y en redes neuronales tradicionales antes de que se popularizaran arquitecturas y funciones de activación más modernas.

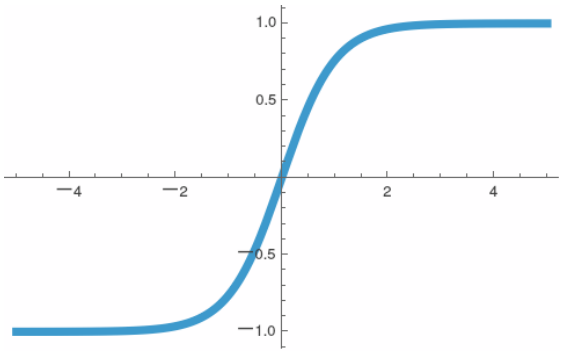



Figura 4.7: Función de activación `tanh`.

#### 4.4.1.5. ReLU (Rectified Linear Unit)

La función ReLU se define por partes:

$$f(x) = \text{máx}(0, x).$$

Activa linealmente las entradas positivas y pone a cero las negativas. Es sencillo y computacionalmente eficiente, favorece la esparcidad de activaciones y reduce en la práctica el problema del gradiente desvanecido en redes profundas. Muy utilizada en capas ocultas de arquitecturas modernas. Tiene la desventaja de poder provocar neuronas "muertas" (gradiente cero para entradas negativas), problema que funciones como Leaky ReLU intentan corregir.

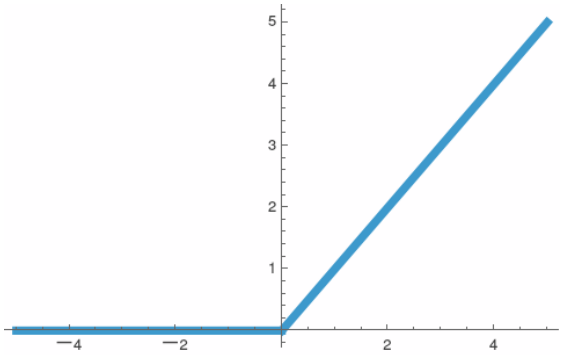



Figura 4.8: Función de activación ReLU.

### 4.4.2. Aprendizaje en redes neuronales

Dada una red neuronal artificial con los parámetros entrenables totalmente configurados, es capaz de realizar su clasificación o predicción de manera correcta. Sin embargo, resulta imposible configurar estos pesos manualmente, por lo que requiere de un "entrenamiento" que ayude a definir los pesos sinápticos. A continuación se define la función de pérdida, optimización y entrenamiento, elementos necesarios para obtener esa configuración específica.

En los primeros modelos, la actualización de los pesos se realizaba a partir del error cuadrático del valor de activación. La regla de actualización se expresaba como:

$$\Delta w_j = -\alpha(c - a)x_j$$

Donde $a$ representa la activación y $\alpha$ la tasa de aprendizaje. Este método permitió superar la no diferenciabilidad de la función escalón en los perceptrones originales [62].

#### 4.4.2.1. Función de pérdida

La función de pérdida se usa para medir el rendimiento del modelo. Calcula la desviación que existe entre el valor predicho y el valor esperado. Esta función puede ser un escalar o un vector que permite aproximar la tasa de error para llegar a una solución óptima [83].

**Definición 16.** *Sea $F_\theta$ una red neuronal artificial y $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$ un conjunto de datos de entrenamiento donde $x^{(i)}$ es la entrada y $y^{(i)}$ es la salida esperada;*

*una función de pérdida es un mapeo que tiene la siguiente forma:*

$$\mathcal{L} : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}_{\geq 0}$$

*Tal que mide la discrepancia entre la predicción del modelo y la salida real:*

$$\mathcal{L}(F_\theta(x^{(i)}, y^{(i)}))$$

Es importante que la función de pérdida sea diferenciable. El gradiente de la función de pérdida permite conocer la dirección hacia la cual debe moverse para disminuir la pérdida.

Las funciones de pérdida se pueden utilizar para regresión y para clasificación. La primera mide el error en valores continuos y se utiliza para estimar valores contables como el precio, tamaño y tiempo. La segunda mide los errores en valores discretos, como la clasificación o la categoría.

Distintas funciones de pérdida priorizan diferentes tipos de error; la elección de la función de pérdida dependerá de la tarea que se desea realizar. Además, la función de pérdida únicamente se aplica a una muestra; para una población de muestras, se utiliza la función de costo, que se define a continuación [83, 84]:

**Definición 17.** *Sea $F_\theta$ una red neuronal artificial y $\mathcal{L}$ la función de pérdida; la función de costo es el promedio de las pérdidas:*

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(F_\theta(x^{(i)}), y^{(i)})$$

#### 4.4.2.2. Optimización

El entrenamiento de una red consiste en ajustar los parámetros entrenables $\theta$ para minimizar la función de pérdida. La optimización consiste en resolver el problema de minimización dado por:

$$\theta^* = \arg\min_{\theta} J(\theta)$$

Esto se puede conseguir mediante diferentes técnicas, siendo el descenso del gradiente la más común. Este método actualiza los parámetros en la dirección opuesta al gradiente de la función de pérdida:

$$\theta_{t+1} = \theta_{(t)} - \eta \nabla_\theta J(\theta^{(t)})$$

donde $\eta > 0$ es la tasa de aprendizaje. Una tasa de aprendizaje alta puede divergir en la solución, mientras que una tasa baja puede tener una convergencia lenta [62].

En la práctica, es común iniciar el entrenamiento con una tasa de aprendizaje relativamente alta y luego reducirla gradualmente a medida que el modelo converge. También se suele incorporar un término de *momentum* para acelerar la convergencia y reducir oscilaciones [62].

#### 4.4.2.3. Entrenamiento

Antes del entrenamiento, las variables de entrada deben normalizarse para ajustarse al rango de activación de la red (por ejemplo, $[0, 1]$ o $[-1, 1]$). Este paso evita que una variable con valores grandes domine la función de agregación y mejora la estabilidad del entrenamiento. Es importante que la normalización preserve la distribución original de los datos y la proporción entre atributos [62].

Finalmente, el entrenamiento consiste en ajustar todos los parámetros $\theta = \{W^{(l)}, b^{(l)}\}$ para minimizar la función de costos $J(\theta)$ en un conjunto de datos de entrenamiento. Este proceso se compone de las siguientes etapas:

1. Se inicializan los parámetros, generalmente de forma aleatoria: $\theta_0 \sim \mathcal{D}_{init}$
2. Se hace la propagación hacia adelante o *forward pass*, es decir, para cada entrada $x^{(i)}$ se evalúa $\hat{y}^{(i)} = F_\theta(x^{(i)})$
3. Se calcula la pérdida: $\mathcal{L}^{(i)} = \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$
4. Se calculan los gradientes propagando la información hacia atrás o *backpropagation* utilizando la regla de la cadena.

   $$\nabla_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \mathcal{L}(F_\theta(x^{(i)}, y^{(i)}))$$

   Se actualizan los parámetros mediante la función de optimización: $\theta_{t+1} = \theta_{(t)} - \eta \nabla_\theta \mathcal{L}(\theta^{(t)})$
5. Se repite este proceso sobre un *dataset* hasta la convergencia de la solución o hasta alcanzar un criterio de parada [62].

   Una vez entrenada la red, puede evaluarse la importancia relativa de las entradas mediante un análisis de sensibilidad. Este procedimiento consiste en:

*a*) Calcular el valor medio de cada entrada.

*b*) Evaluar la salida de la red con todos los valores medios.

*c*) Modificar una entrada a la vez (aumentándola o disminuyéndola) y medir el cambio en la salida.

*d*) Repetir el proceso para cada variable de entrada.

De esta manera, se identifica qué variables influyen más en el resultado, lo cual aporta interpretabilidad al modelo [62].

La elección de la topología de la red, en particular el número de neuronas ocultas, suele determinarse empíricamente. Una regla práctica consiste en comenzar con un número de neuronas ocultas similar al número de entradas y ajustar a partir del rendimiento obtenido. Otras heurísticas recomiendan iniciar con el doble de nodos de entrada y reducir gradualmente si se detecta sobreajuste [62].

En resumen, el entrenamiento de la red neuronal es un proceso iterativo de optimización que consiste en:

1. Propagar los datos de entrada por la red.
2. Comparar la salida con el valor real usando una función de pérdida.
3. Propagar el error hacia atrás permite obtener gradientes.
4. Actualizar los parámetros con un algoritmo de optimización.
5. Repetir hasta que el modelo aproxime adecuadamente la función deseada.

## 4.5. Tipos de redes neuronales

Las redes neuronales pueden clasificarse atendiendo a diferentes criterios, ya sea por el número de capas, el tipo de conexiones, el grado de conectividad o la arquitectura general. En este trabajo se han seleccionado dos tipos principales, de manera enunciativa, mas no limitativa, que son especialmente relevantes para su eventual implementación en un autómata celular neuronal: las redes neuronales convolucionales y las recurrentes. La razón de esta elección es la siguiente:

Por un lado, las evoluciones de un autómata celular ocurren de forma secuencial en el tiempo, es decir, las entradas y salidas de la red corresponden a estados del

espacio celular y, en algunos casos, las reglas pueden tener una relación temporal o reversible. De ahí que las redes recurrentes resulten particularmente apropiadas.

Por otro lado, como se explicará más adelante, se ha demostrado que los autómatas celulares y las redes neuronales convolucionales tienen algunas similitudes, especialmente porque el *kernel* de la red convolucional es similar a la vecindad del autómata celular.

### 4.5.1. Redes neuronales convolucionales

Las redes neuronales convolucionales constituyen una clase de redes profundas cuya estructura está diseñada para explotar la topología de los datos que tienen forma de rejilla. Por ejemplo, imágenes en dos dimensiones.

Una red neuronal convolucional consta de una capa de entrada, varias capas ocultas (que pueden incluir capas de convolución, normalización, *pooling*, activación, etc.) y una capa de salida.

Se asume un volumen de entrada con dimensiones ($H \times W \times C$), es decir, altura $H$, anchura $W$ y $C$ canales (por ejemplo, RGB en imágenes). La red maneja este volumen usando filtros deslizantes. Aunque normalmente no se describen en términos de estados discretos de autómatas, se puede entender que cada neurona de una capa representa un estado continuo real (por ejemplo, una activación $a \in \mathbb{R}$).

El elemento clave es su operación de convolución: un filtro (o *kernel*) de tamaño $k \times k$ se desliza sobre la entrada, compartiendo pesos y produciendo mapas de activación. Se puede ver como una vecindad local de la neurona que influye en su valor de activación que se aplica en cada capa de convolución:

$$Y = f(X * K + b)$$

Donde $X$ es el volumen de entrada, $K$ es el filtro (o filtros) de convolución, $b$ es el sesgo y $f$ es una función de activación. A continuación pueden seguir operaciones de *pooling* y capas totalmente conectadas para producir la salida final [85].

Se aplica principalmente en clasificación de imágenes, segmentación, reconocimiento de video y procesamiento de audio. En el contexto de los autómatas celulares, se puede ver que el filtro de la red neuronal convolucional es similar a la vecindad del autómata. Esto permite entenderlo como una operación de actualización local que se repite en un espacio organizado, igual que en un autómata

celular.

### 4.5.2. Redes neuronales recurrentes

Las redes neuronales recurrentes son un tipo de redes que tienen conexiones que vuelven sobre sí mismas. Esto permite que la información de momentos anteriores afecte el procesamiento que se está realizando en el presente.

Una red neuronal recurrente trabaja con una serie de entradas $(x_1, x_2, \ldots, x_T)$ y genera una serie de salidas $(y_1, y_2, \ldots, y_T)$. En cada momento, la red tiene un estado oculto $h_t$ que se actualiza de la siguiente manera:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h),$$
$$y_t = g(W_{hy}h_t + b_y),$$

Donde $W_{xh}$, $W_{hh}$ y $W_{hy}$ son matrices de pesos, $b_h$ y $b_y$ son sesgos, y $f$, $g$ son funciones de activación.

La red funciona en una secuencia de tiempo, donde cada momento puede verse como una çapa virtual.º una repetición del autómata. En cada instante $t$, el estado oculto $h_t$ muestra la "memoria.º el contexto acumulado hasta ese momento.

La vecindad temporal está dada por la conexión de la unidad de tiempo $t$ con la anterior $t-1$ (y en variantes bidireccionales, también con $t+1$). Es decir, la salida de un paso se inserta como parte de la entrada del siguiente paso.

Como se mostró más arriba, la actualización recursiva del estado oculto actúa como la función de transición temporal de la red. La red integra información histórica para producir la salida en cada paso de tiempo.

Estas redes se utilizan frecuentemente en el procesamiento de lenguaje natural, reconocimiento de voz, series temporales, traducción automática y otras tareas en las que el orden temporal de los datos es esencial [86].

Dado que un autómata celular evoluciona en pasos discretos de tiempo y cada uno puede depender del estado previo del sistema, las redes neuronales recurrentes aportan un modelo natural para capturar dependencias temporales entre estados sucesivos del autómata. Esto resulta especialmente útil si la regla del autómata celular tiene memoria o se desea que aprenda dinámicas espacio-temporales.

En conclusión, mientras que las redes neuronales convolucionales capturan la estructura espacial y la vecindad local del sistema, las redes neuronales recurrentes modelan su evolución y persistencia temporal. La integración de ambas capacidades permite que un autómata celular neuronal no solo emule reglas estables, sino que aprenda dinámicas complejas a partir de datos, superando las limitaciones de las reglas de transición diseñadas manualmente.

# Bibliografía


[1] W. MathWorld, "Cellular automaton," n.d., consultado el 28 de Marzo del 2025. [Online]. Available: https://mathworld.wolfram.com/CellularAutomaton.html

[2] H. Zenil and G. J. Martínez, "Cellular automata," *Scholarpedia*, vol. 19, no. 4, p. 53227, 2024. [Online]. Available: http://www.scholarpedia.org/article/Cellular_automata

[3] D. A. R. Gómez, "Descripción y aplicaciones de los autómatas celulares," 2011, consultado el 27 de Marzo del 2025. [Online]. Available: https://delta.cs.cinvestav.mx/~mcintosh/cellularautomata/Summer_Research_files/Arti_Ver_Inv_2011_DARG.pdf

[4] M. O. Domínguez, "Redes neuronales artificiales," *Ingenio y Conciencia Boletín Científico de la Escuela Superior Ciudad Sahagún*, vol. 12, no. 23, pp. 38–44, 2025. [Online]. Available: https://doi.org/10.29057/escs.v12i23.14132

[5] D. J. Matich, "Redes neuronales: Conceptos básicos y aplicaciones." n.d., consultado el 28 de Marzo del 2025. [Online]. Available: https://www.frro.utn.edu.ar/repositorio/catedras/quimica/5_anio/orientadora1/monograias/matich-redesneuronales.pdf

[6] D. Pub, "Growing cellular automata," 2020, consultado el 27 de Marzo del 2025. [Online]. Available: https://distill.pub/2020/growing-ca/

[7] *Emerging Trends in Applications and Infrastructures for Computational Biology, Bioinformatics, and Systems Biology*. Elsevier, 2016, accedido el 13 de junio de 2025. [Online]. Available: https://doi.org/10.1016/c2015-0-01779-8

[8] S. E. Yacoubi, T. Plénet, S. Dridi, F. Bagnoli, L. Lefèvre, and C. Raïevsky, "Some control and observation issues in cellular automata," *Complex Systems*,

vol. 30, no. 3, pp. 391–413, Sep. 2021, accedido el 14 de junio de 2025. [Online]. Available: https://doi.org/10.25088/complexsystems.30.3.391

[9] W. Gilpin, "Cellular automata as convolutional neural networks," *Physical Review E*, n.d., consultado el 29 de Marzo del 2025. [Online]. Available: https://journals.aps.org/pre/abstract/10.1103/PhysRevE.100.032402

[10] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, 2020, https://distill.pub/2020/growing-ca.

[11] ——, "Growing neural cellular automata," *Distill*, 2020, https://distill.pub/2020/growing-ca.

[12] Instituto Nacional de Estadística e Informática, "¿qué es la teoría general de sistemas?" INEI - Sub-Jefatura de Informática, Lima, Perú, Manual, 1999. [Online]. Available: https://cmapspublic.ihmc.us/rid=1G8TFJM82-16RKYJR-M62/TGS.pdf

[13] M. Arnold-Cathalifaud and F. Osorio, "Introducción a los conceptos básicos de la teoría general de sistemas," *Cinta de Moebio*, no. 3, mar 1998. [Online]. Available: https://www.redalyc.org/articulo.oa?id=10100306

[14] S. C. Milagros. (2024, jul) Ludwig von bertalanffy y su teoría general de los sistemas. [Online]. Available: https://neuro-class.com/bertalanffy-y-su-tgs/

[15] R. R. Solano, "Orígenes de la teoría de sistemas," Universidade Federal do Paraná (UFPR), Material de apoyo, 2019. [Online]. Available: https://www.inf.ufpr.br/urban/2019-1_220/220_Ler_Estudar_Discutir/Solano__Origines_Teoria_Sistemas.pdf

[16] A. M. Caraballo Payares, *Evaluación de la gestión administrativa, financiera y comercial de las cooperativas asociadas a CONFECOOP Caribe*. eumed.net, 2013. [Online]. Available: https://www.eumed.net/libros-gratis/2013a/1314/sistemas.html

[17] Equipo editorial Concepto. (2025) Sistema - características, tipos y ejemplos. Accedido el 8 de nov. de 2025. [Online]. Available: https://concepto.de/sistema

[18] J. D. Monsalve Ospina, "La dualidad de sistemas: sistemas abiertos y cerrados en la teoría general de sistemas," 2023, accedido el 8 de nov. de 2025. [Online]. Available: https://udecolombia.edu.co/wp-content/uploads/2024/12/Monsalve-JD.-2023_TGS-Sistemas-abiertos-y-cerrados.pdf

[19] M. S. Rojas, "Determinísticos y probabilísticos," 2018, accedido el 8 de nov. de 2025. [Online]. Available: https://www.scribd.com/document/380696672/Deterministicos-y-Probabilisticos

[20] Anónimo, "Teoría general de sistemas: Sistemas Estáticos - Dinámicos," 2025, accedido el 9 de nov. de 2025. [Online]. Available: https://8d8a87251d.cbaul-cdnwnd.com/b3abc3d20220b43c1b18cf3768b8f6e1/200000090-2218722189/Sistemas%20Estaticos%20-%20Dinamicos-0.pdf

[21] A. O. Cordero, "Introducción a sistemas dinámicos," 2025, accedido el 8 de nov. de 2025. [Online]. Available: https://umi.cinvestav.mx/Portals/UMI_LAB/Maestria%20material%20de%20apoyo/SeÃśales%20y%20sistemas%201.pdf

[22] E. E. Alberto, H. E. Tenoch, and E. E. V. Romero, "Exploración de universos discretos con autómatas celulares en dos dimensiones," 2021, accedido el 8 de nov. de 2025. [Online]. Available: https://www.comunidad.escom.ipn.mx/genaro/Papers/Thesis_files/tesisEricEduardoAlbertoTT2019-B067.pdf

[23] I. Altadill Simón and D. R. García Belinchón, "La teoría del caos," 2025, accedido el 8 de nov. de 2025. [Online]. Available: https://repositorio.unican.es/xmlui/bitstream/handle/10902/30650/AltadillSimonIgnacio.pdf

[24] J. Torres, "Introducción a los sistemas complejos," 2025, accedido el 8 de nov. de 2025. [Online]. Available: https://www.ugr.es/~jtorres/tema_1_sistemas_complejosv2.pdf

[25] I. Zelinka, A. Sanayei, H. Zenil, and O. E. Rössler, *How Nature Works*. Heidelberg: Springer International Publishing, 2014.

[26] I. Zelinka and J. Burguillo, "Complex systems," 01 2018, pp. 11–34.

[27] R. V. Solé and S. C. Manrubia, *Orden y caos en sistemas complejos*. Barcelona: Edicions UPC, 1996.

[28] S. Moriello. (2003) Sistemas complejos, caos y vida artificial. Accedido el 23 de sept. de 2025. [Online]. Available: https://www.redcientifica.com/doc/doc200303050001.html

[29] Gamco. (2024) Simulación: Definición y aplicaciones en el ámbito empresarial. Consultado el 28 de marzo de 2026. [Online]. Available: https://gamco.es/glosario/simulacion/

[30] L. R. Izquierdo, J. M. Galán Ordax, J. I. Santos, and R. del Olmo Martínez, "Modelado de sistemas complejos mediante simulación basada en agentes y mediante dinámica de sistemas," *Empiria. Revista de Metodología de Ciencias Sociales*, no. 16, pp. 85–112, oct 2008.

[31] A. Hoekstra, J. Kroc, and P. Sloot, "Introduction to modeling of complex systems using cellular automata," in *Simulating Complex Systems by Cellular Automata*, 05 2010, vol. 2010, pp. 1–16.

[32] J. Damerow. (2010) John von Neumann's cellular automata. Embryo Project Encyclopedia. Accedido el 23 de julio de 2025. [Online]. Available: https://embryo.asu.edu/pages/john-von-neumanns-cellular-automata

[33] E. W. Weisstein. (2002) Cellular automaton. MathWorld – A Wolfram Resource. Accedido el 23 de julio de 2025. [Online]. Available: https://mathworld.wolfram.com/CellularAutomaton.html

[34] S. Wolfram, "Cellular automata as simple self-organizing systems," California Institute of Technology, Technical Report CALT-68-938, 1982. [Online]. Available: https://content.wolfram.com/sw-publications/2020/07/cellular-automata-self-organizing-systems.pdf

[35] F. S. Caparrini. (2016) Autómatas celulares. Universidad de Sevilla. Accedido el 24 de septiembre de 2025. [Online]. Available: https://www.cs.us.es/~fsancho/Blog/posts/Automatas_Celulares.md.html

[36] K. Bhattacharjee, "Cellular automata: Reversibility, semi-reversibility and randomness," PhD Thesis, Indian Institute of Engineering Science and Technology, Shibpur, 2019, accedido el 23 de agosto de 2025. [Online]. Available: https://arxiv.org/abs/1911.03609

[37] H. Sayama. (2024) Definition of cellular automata. LibreTexts / Binghamton University. Accedido el 24 de septiembre de 2025. [Online]. Available: https://math.libretexts.org/Bookshelves/Scientific_Computing_Simulations_and_Modeling/Introduction_to_the_Modeling_and_Analysis_of_Complex_Systems_(Sayama)/11%3A_Cellular_Automata_I__Modeling/11.01%3A_DeïňĄnition_of_Cellular_Automata

[38] A. Ilachinski, *Cellular Automata: A Discrete Universe*. Singapore: World Scientific Publishing, 2001.

[39] J. von Neumann and A. W. Burks, *Theory of Self-Reproducing Automata*, A. W. Burks, Ed. Urbana, Illinois: University of Illinois Press, 1966. [Online]. Available: https://cba.mit.edu/events/03.11.ASE/docs/VonNeumann.pdf

[40] Y. W. Kwon, *Multiphysics and Multiscale Modeling: Techniques and Applications*, 1st ed. CRC Press, 2015.

[41] Conway's game of life. LifeWiki. Accedido el 24 de septiembre de 2025. [Online]. Available: https://conwaylife.com/wiki/Conway%27s_Game_of_Life

[42] S. Wolfram, *A New Kind of Science*. Champaign, Illinois: Wolfram Media, 2002.

[43] T. Tyler. (2018) Cellular automata - the margolus neighbourhood. Accedido el 24 de septiembre de 2025. [Online]. Available: https://cell-auto.com/neighbourhood/margolus

[44] O. Ureña Ponce, "Diseño de un procesador de autómatas celulares para la simulación de sistemas complejos," Master's thesis, Universidad de Guadalajara, Guadalajara, México, 2015. [Online]. Available: https://riudg.udg.mx/bitstream/20.500.12104/79915/1/MCUCEI01068FT.pdf

[45] G. J. Martinez, A. Adamatzky, and J. Seck-Tuoh-Mora, "Some notes about the game of life cellular automaton," in *Emergent Computation*. Springer, 2022, pp. 93–104. [Online]. Available: https://www.comunidad.escom.ipn.mx/genaro/Papers/Papers_on_CA_files/2022SomeNotesGoL.pdf

[46] E. W. Weisstein. (2002) Totalistic cellular automaton. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/TotalisticCellularAutomaton.html

[47] ——. (2003) von Neumann neighborhood. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/vonNeumannNeighborhood.html

[48] Wikipedia contributors. (2016) Cellular automaton — wikipedia, the free encyclopedia. Accedido el 12 de noviembre de 2025. [Online]. Available: https://en.wikipedia.org/wiki/Cellular_automaton

[49] M. Thoma. (2020, nov) John Conway's game of life - an introduction to cellular automata. Accedido el 11 de noviembre de 2025. [Online]. Available: https://beltoforion.de/en/game_of_life

[50] M. Sipper, "Paul Rendell: Turing machine universality of the Game of Life," *Genetic Programming and Evolvable Machines*, vol. 18, no. 1, pp. 115–117, 2017, accedido el 11 de noviembre de 2025. [Online]. Available: https://www.moshesipper.com/pubs/rendell_review.pdf

[51] P. Rendell. (2000) Turing machine in Conway's Game of Life. Rendell's Attic. Accedido el 29 de marzo de 2026. Imagen disponible en http://rendell-attic.org/gol/turing_js_r.gif. [Online]. Available: http://rendell-attic.org/gol/tm.htm

[52] E. W. Weisstein. (2002) Rule 30. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/Rule30.html

[53] ——. (2002) Rule 110. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/Rule110.html

[54] ——. (2002) Rule 90. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/Rule90.html

[55] G. J. Martínez. (2000) Teoría del campo promedio en autómatas celulares similares a "the game of life". Accedido el 8 de septiembre de 2025. [Online]. Available: https://delta.cs.cinvestav.mx/~mcintosh/comun/tesismaestria/genaro/thesis.html

[56] G. J. Martinez. (2013) A note on elementary cellular automata classification. [Online]. Available: https://arxiv.org/abs/1306.5577

[57] K. C. II and S. Yu, "Undecidability of ca classification schemes," *Complex Systems*, vol. 2, pp. 177–190, 1988.

[58] D. Grattarola, L. Livi, and C. Alippi, "Learning graph cellular automata," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 20 983–20 994. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/af87f7cdcda223c41c3f3ef05a3aaeea-Paper.pdf

[59] P. Gács, "Probabilistic cellular automata with Andrei Toom," *Brazilian Journal of Probability and Statistics*, vol. 38, no. 2, 2024, accedido el 23 de julio de 2025. [Online]. Available: https://doi.org/10.1214/24-bjps605

[60] H. Hartman, P. Tamayo, and W. Klein, "Inhomogeneous cellular automata and statistical mechanics," *Complex Systems*, vol. 1, no. 2, 1987. [Online]. Available: https://content.wolfram.com/uploads/sites/13/2018/02/01-2-2.pdf

[61] L. Becker. (2023) Continuous cellular automata – HEGL. Heidelberg Experimental Geometry Lab. Accedido el 23 de julio de 2025. [Online]. Available: https://hegl.mathi.uni-heidelberg.de/continuous-cellular-automata

[62] D. J. Matich, *Redes Neuronales Artificiales: Conceptos Básicos y Aplicaciones*, Universidad Tecnológica Nacional (UTN), Facultad Regional Rosario, Argentina, 2001, consultado el 14 de abril de 2026. [Online]. Available: https://www.frro.utn.edu.ar/repositorio/catedras/quimica/5_anio/orientadora1/monograias/matich-redesneuronales.pdf

[63] C. Manning, "Artificial intelligence definitions," https://hai-production.s3.amazonaws.com/files/2020-09/AI-Definitions-HAI.pdf, 2020, consultado el 23 de septiembre de 2025.

[64] Google Cloud, "¿qué es la inteligencia artificial o ia?" 2025, consultado el 20 de marzo de 2025. [Online]. Available: https://cloud.google.com/learn/what-is-artificial-intelligence?hl=es-419

[65] J. McCarthy, "What is artificial intelligence?" 2007, consultado el 20 de marzo de 2025. [Online]. Available: http://jmc.stanford.edu/articles/whatisai/whatisai.pdf

[66] Oracle, "¿qué es el machine learning?" 2024, consultado el 20 de marzo de 2025. [Online]. Available: https://www.oracle.com/mx/artificial-intelligence/machine-learning/what-is-machine-learning/

[67] Google Cloud. (s.f.) ¿qué es el aprendizaje automático? tipos y usos. Consultado el 07 de abril de 2025. [Online]. Available: https://cloud.google.com/learn/what-is-machine-learning?hl=es-419

[68] D. Bergmann. (s.f.) ¿qué es el aprendizaje automático? Consultado el 07 de abril de 2025. [Online]. Available: https://www.ibm.com/mx-es/think/topics/machine-learning

[69] A. Main, "What is ai training?" https://www.linkedin.com/pulse/what-ai-training-andy-main-2f4oe/, 2023, consultado el 07 de abril de 2025.

[70] Oracle. (2024) ¿qué es el big data? Consultado el 28 de abril de 2025. [Online]. Available: https://www.oracle.com/mx/big-data/what-is-big-data/

[71] Google Cloud. (s.f.) ¿qué es el aprendizaje no supervisado? Consultado el 07 de abril de 2025. [Online]. Available: https://cloud.google.com/discover/what-is-unsupervised-learning?hl=es-419

[72] Amazon Web Services. (s.f.) ¿qué es el aprendizaje mediante refuerzo? Consultado el 07 de abril de 2025. [Online]. Available: https://aws.amazon.com/es/what-is/reinforcement-learning/

[73] Adobe. (s.f.) Machine learning — definition, models, and applications. Consultado el 07 de abril de 2025. [Online]. Available: https://business.adobe.com/ar/products/real-time-customer-data-platform/what-is-machine-learning.html

[74] Clínica Universidad de Navarra, "Qué es la neurona," 2023, consultado el 17 de septiembre de 2025. [Online]. Available: https://www.cun.es/diccionario-medico/terminos/neurona

[75] P. Bordoli, "La neurona," *Repositorio Académico*, 2023, consultado el 17 de septiembre de 2025.

[76] brgfx, "Freepik graphic resource," http://www.freepik.com, 2025, consultado el 17 de septiembre de 2025.

[77] S. Mhatre. (2020) What is the relation between artificial and biological neuron? Consultado el 17 de septiembre de 2025. [Online]. Available: https://saurabhnativeblog.medium.com/what-is-the-relation-between-artificial-and-biological-neuron-18b05831036

[78] Y. Lao León, A. Rivas Méndez, M. Pérez Pravia, and F. Marrero Delgado, "Procedimiento para el pronóstico de la demanda mediante redes neuronales artificiales," *Ciencias Holguin*, vol. 23, pp. 43–59, 2017, consultado el 17 de septiembre de 2025.

[79] Google, "Neural networks: Activation functions," s.f., consultado el 17 de septiembre de 2025. [Online]. Available: https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions

[80] Amazon Web Services, "¿qué es una red neuronal?" s.f., consultado el 17 de septiembre de 2025. [Online]. Available: https://aws.amazon.com/es/what-is/neural-network

[81] Telefónica Tech, "Las matemáticas del machine learning: Redes neuronales (parte i)," 2019, consultado el 17 de septiembre de 2025. [Online]. Available: https://telefonicatech.com/blog/las-matematicas-del-machine-learning-redes-neuronales-parte-i

[82] IBM Corporation, *Neural networks - IBM SPSS Modeler*, IBM, 2021, consultado el 14 de abril de 2026. [Online]. Available: https://www.ibm.com/docs/es/spss-modeler/saas?topic=networks-neural-model

[83] IBM, "¿qué es la función de pérdida?" s.f., consultado el 24 de julio de 2025. [Online]. Available: https://www.ibm.com/mx-es/think/topics/loss-function

[84] Ultralytics, "Función de pérdida en ia/ml explicada," 2025, consultado el 24 de julio de 2025. [Online]. Available: https://www.ultralytics.com/es/glossary/loss-function

[85] A. Amidi and S. Amidi. (2025) Cs 230 - convolutional neural networks cheatsheet. Stanford University. Consultado el 13 de noviembre de 2025. [Online]. Available: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks

[86] ——. (2025) Cs 230 - recurrent neural networks cheatsheet. Stanford University. Consultado el 13 de noviembre de 2025. [Online]. Available: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks